



M362 Unit 10
UNDERGRADUATE COMPUTING

Developing concurrent distributed systems



Security

Unit **10**

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1598 9

1.1

CONTENTS

1	Introduction	6
1.1	The aims of this unit	7
2	Security concepts	8
2.1	Distributed threats	8
2.2	The nature of the threat	9
2.3	Security services	10
2.4	Policies	11
3	Cryptography	14
3.1	Java Cryptography Architecture	14
3.2	Ciphers	15
3.3	Strength of ciphers	18
3.4	Symmetric cryptography	21
3.5	Public key cryptography	22
3.6	Authentication	25
3.7	Hash functions	29
3.8	Stream ciphers	31
3.9	Choice of system	32
4	Protocols and networking	34
4.1	General issues	34
4.2	IPv6	35
4.3	Secure Sockets Layer	36
4.4	Firewalls	37
5	An overview of Java platform security features	40
5.1	Java language features	40
5.2	The Java Virtual Machine	42
5.3	Authorisation	44
6	Java authentication and authorisation	51
6.1	Principals and subjects	51
6.2	Login context	52
6.3	An example authentication	54

Continued over page

7	Containers and authorisation	58
7.1	Overview of container security	58
7.2	Authentication mechanisms	61
7.3	Authorisation mechanisms	62
8	Summary	66
	Glossary	68
	References	73
	Index	74



M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1

Introduction

In this block, we consider issues that cut across the tiers of a distributed system, beginning with security. *Unit 11* will look at techniques particularly relevant to heterogeneous systems, and compare Java EE with the .NET platform. *Unit 12* discusses mobile systems.

Distributed computing is now fundamental to the operation of many businesses, and security is essential not only in electronic commerce, but also in such diverse applications as car security systems, on-demand media (for example, pay-television) and mobile telephony for voice encryption and identification of individual handsets. Companies' computer systems, staff, data and communications are key business assets that must be protected.

Broadly, the aims of security techniques are to:

- ▶ protect assets from damage;
- ▶ detect if assets have been tampered with, preferably also identifying how and when this happened, and who was responsible;
- ▶ recover assets after an attack.

Techniques to achieve these aims are not exclusively located in any particular tier of an enterprise system. Indeed, in the context of a discussion of security, the word **system** means not only the application software, but also computer hardware and peripheral devices, operating system software, the users, management and other individuals. We hope it will be clear from the context when a more restricted meaning is intended.

Thus, security mechanisms may be seen as lying on a continuum from user to software to hardware. A security system designer must decide where to apply security measures on this scale. Security mechanisms towards the hardware end of the scale tend to be more generic and restrict higher-layer operations. This has an advantage of simplicity, but does not provide a fine-grained control. Most likely, a combination of measures at various layers is required – some generic, some oriented towards data, some towards operations and others towards users. Some measures might be said to protect the system from its users, others to protect the users from untrusted code. However, it is also worth noting that a measure can often be subverted by bypassing it and accessing its underlying layer directly.

Clearly, we will not be able to discuss all these factors in depth here, and we will not discuss some of the standard technologies, including virus and spam protection, with which you should already have some familiarity. Nor will we discuss hardware measures in general, though we briefly discuss firewalls. These are requirements for any system's defences. Rather we will concentrate on security mechanisms as seen in core Java and in Java EE.

1.1 The aims of this unit

This unit will discuss:

- ▶ the main security services in enterprise systems and some of the threats to them (Section 2);
- ▶ cryptography and its role in protecting data and communications (Sections 3 and 4);
- ▶ how Java provides a robust and flexible security environment (Section 5);
- ▶ authorisation and authentication mechanisms for Java enterprise systems (Sections 6 and 7).

Apart from reading and exercises, this unit also contains a relatively large number of practical activities for which you may need to allow extra study time.

2

Security concepts

In this section, we will introduce some standard security terminology and describe the high-level services that security mechanisms are expected to provide to systems.

We will also briefly consider security measures that can be applied to users and the idea of a firewall, which is typically a hardware measure.

We begin with an overview of the various kinds of threat a system may be open to, whether perpetrated by attackers or inadvertently caused, as a way of setting the stage for countermeasures.

2.1 Distributed threats

There are various **vectors**, that is, routes, by which computers may be subject to threats of one kind or another. Vectors include portable storage media as well as email and other forms of networked communications. Prior to the wide use of the internet, the most common vector was the floppy disk, but networks are now the most common route of attack. Security holes in application and operating system software may be exploited by an attacker sending a malformed file or URL, for example. (We will not be concerned with the details of how such exploits work in practice.)

System administrators also need to be aware of who the enemy is. This and knowledge of the main threats will assist in targeting efforts towards the greatest danger.

Here is a broad categorisation of the kinds of attacker that systems need to be protected from.

- ▶ **Hackers** are typically people who enjoy the challenge of defeating security measures, not necessarily with malicious intent. On the more sinister side, hacking may include activities such as keystroke logging, password harvesting, identity theft, or taking over use of a PC, for example using it to send spam.
- ▶ Unethical businesses may seek information about a user's shopping or browsing habits, without their knowledge. In **click fraud**, pay-per-click charges (that is, charges applied to each visit) to online advertisements are generated fraudulently. This can be done by a person repeatedly visiting a page, or a program imitating a user, for example. Competitors may also attempt to gain access to confidential information, such as source code for a new product, or strategic plans of other businesses.
- ▶ Bulk-mailers (spammers and others making illegitimate or fraudulent offers) flood computer systems with unwanted traffic and waste users' time. Fraudsters usually attempt to gain access to confidential information, steal an individual's identity, or misuse an individual's trust.
- ▶ Employees, other system users, or members of the public may also be a threat to a system, through either deliberate effort or unintended misuse.

A common threat at the enterprise level is extortion, frequently through the suggestion that confidential data (customer credit card numbers or medical information, for example) will be exposed. You will be able to find many examples of such attacks with a little research.

Google settled a click-fraud case for about \$90m (£48m) in July 2006.

In 2006, according to the BBC, identity theft cost the UK economy £1.7 billion.

US law now requires individuals to be notified if their details are compromised.

According to the Gartner group (Hunter, 2003), employees are the biggest threat and are responsible for 70% of unauthorised access and for 95% of intrusions resulting in financial loss.

Often intrusion into a system is as a result of an employee assisting a caller posing as an employee or customer in need of information. This is an example of the user-based attack, sometimes called **social engineering**; however, this term covers many kinds of attack, from watching a user enter a password to tricking a user into compromising the system, for example by revealing information or deleting a file. Other tricks of 'engineering' include causing panic and haste, for example by claiming that a large bill has been charged to a user's account or that the account has been compromised.

It is also common to misdirect users to fake websites through a variety of 'spoofing' techniques. For example, in **homograph attacks**, similar looking characters in different languages can lead to identical looking domain names that are actually distinct under international domain name support. A less subtle approach is to register a visually similar domain name such as microsf.com or rnicrosoft.com.

Control system attack

In November 2001, 49-year-old Vitek Boden was sentenced to two years in prison for using the Internet, a wireless radio and stolen control software to release up to one million litres of sewage into the river and coastal waters of Maroochydore in Queensland, Australia.

Boden, who had been a consultant on the water project, conducted the attack in March 2000 after he was refused a full-time job with the Maroochy Shire government. He had attempted to gain access to the system 45 times, and his last attempt proved successful, allowing him to release raw sewage into the waterways.

'Marine life died, the creek water turned black and the stench was unbearable for residents,' said Janelle Bryant, investigations manager for the Australian Environmental Protection Agency.

(Lemos, 2002)

2.2 The nature of the threat

At the heart of these seemingly diverse security issues in distributed computing is communication of various kinds between computer users. Comparing these communications to paper documents passed between individuals helps to highlight a number of issues as, traditionally, paper-based communications provided some assurances that are relatively hard to provide in electronic communication.

- ▶ Copies or fabrications of paper documents are usually detectable. Electronic copies of communications may be indistinguishable from one another.
- ▶ Alterations to paper documents are detectable by close examination. There may be no evidence that an electronic communication has been altered.
- ▶ The contents of a document and the means of telling who wrote it are clearly separated on paper. The writer may have signed the document or attached a seal, or a trusted individual may have carried the document from the author. For an electronic communication, it is harder to separate authentication from content.

Figure 1 illustrates classes of interference with normal communication, as suggested by Stallings (1999). These communications may take place over a network, but the idea of source and destination expressed here is a broad one. For example, the source may be a file, and the destination a user of that file, on the same system.



Keeping your personal computer safe.

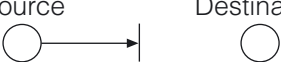
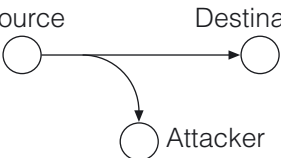
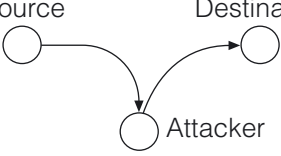
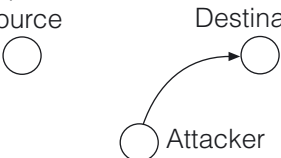
(a) Normal operation Source Destination 	Normal operation implies secure distributed or local communications.
(b) Interruption Source Destination 	Interruption implies that an asset is unusable, or unavailable (perhaps only for a period of time) when required. In the context of network communications, we normally call this a denial of service. For example, a denial of service attack on a website could flood the website with requests for pages so that the server is kept too busy to allow legitimate users to access pages.
(c) Interception Source Destination 	Interception implies that an unauthorised party has gained some access to an asset. This may result in the asset's contents being directly disclosed or other information being obtained (for example by an analysis of the asset or an analysis of the information flow, with the contents of the asset or communication not being disclosed). Another danger is that the attacker replays intercepted information in some way; for example, to gain access to a system, or cause a previously performed transaction (such as a bank transfer) to be repeated.
(d) Modification Source Destination 	Modification is interception with alteration to an asset. For example, the account number of a bank transfer could be changed by an attacker, who may also impersonate the original sender.
(e) Fabrication Supposed source Destination 	Fabrication is the creation of an unauthorised new object in the system.

Figure 1 Classes of security threat

In all these scenarios an outside party may play a part, but whether or not this is the case, security systems must be able to document the activity on a given computer network so that the flow of information can be tracked, and actions can be attributed to particular parties.

An intercepting party is sometimes known as a **middle person** (alternatively termed a ‘man-in-the-middle’ or ‘person-in-the-middle’). A middle-person attack is a particular danger in wireless networks where the attacker need not have access to a physical network.

The first release of the Google desktop search was vulnerable to middle-person attacks.



Google desktop search (first release) security issue.

2.3 Security services

In implementing security, businesses and software manufacturers may subscribe to published standards in which various security services are defined, sometimes with

slightly different connotations according to the standard. The main services required by enterprise systems (and Java EE products) are the following.

- 1 **Confidentiality** ensures that the assets in a computer system are accessible only to authorised users. This includes file access protections and secure communications, to protect users from inadvertently sharing sensitive information, as well as cryptographic services.
- 2 **Integrity** controls are concerned with detecting malicious or accidental alteration of data.
- 3 **Non-repudiation** requires that the sender of a message must not be able to plausibly deny sending it, and the receiver must not be able to deny receiving it. In other words, users must be accountable for their actions.
- 4 **Auditing** captures a secure record of security-related events for the purposes of non-repudiation and recovery, and also to determine the effectiveness of security systems.
- 5 **Authentication**, also known as **identification**, is the way a system determines who a user is and ensures that actions are correctly associated with individuals.
- 6 **Authorisation** describes how a system determines what a user or program is allowed to do. Systems need to be robust enough to deal with users' mistakes or deliberate actions. (The Java EE specification also refers to this as access control.)

The enforcement of these services helps to ensure the **availability** of a system; that is, that assets are accessible by authorised users when needed.

Throughout this unit, we will be discussing ways in which these services are implemented.

2.4 Policies

Before considering more software-oriented approaches, we want to mention some security measures that may be applied at the user and hardware ends of the security continuum.

Examples of policies enforced by a network administrator may include:

- ▶ require passwords to be a certain length or to contain non-alphabetic characters, or to be changed regularly;
- ▶ ensure that user accounts have no more privileges than required;
- ▶ apply a regular cycle of system backups;
- ▶ enforce update or patch cycles;
- ▶ deploy virus protection and anti-spyware systems;
- ▶ install and configure a firewall;
- ▶ have users sign statements of legitimate use of computer resources.

A security administrator can employ **threat-monitoring** techniques, in other words, keep a watchful eye on the system, and regularly assess its use.

An **audit log** is a record of system use that can be used for threat-monitoring and recovery. This might include checking on how often failed logins are occurring on an account, or scanning for various categories of potentially dangerous or suspicious files. Administrators may look for unexpected activity, altered programs, out-of-date programs, or incorrect protection settings on folders and files. **Packets** travelling through a network can be analysed to see if they should be allowed to reach their intended destinations.

The Open University
course M886
*Information security
management*
discusses
management issues for
security in detail.

An audit log may be analysed for unusual user behaviour or evidence of accounts having been compromised. This may help to reconstruct an attack, which can help to recover an asset after the fact. Logs can also be used as evidence in cases where there is a dispute over when, where, or how something happened in the system. Such evidence may be used to support action against a perpetrator.

Auditing is also a service provided by components in various Java EE implementations.

Obscurity

Knowing that many attackers will restrict their efforts to the most obvious targets, an administrator may employ a 'honeypot', which is typically a service such as a SQL server or FTP server. A honeypot behaves like the real thing, in its default location, but is actually a decoy, with the real service deployed elsewhere. This can be a way of monitoring threats, but also is often sufficient to move attackers onto another more obvious target.

Other examples of obscuring the operation of a system include hiding details of the design or implementation of software, or hiding the workings of specific components (such as the details of how an encryption algorithm is performed).

Intuitively, hiding such information would seem a good way to promote security; however, the idea that such actions alone (termed **security through obscurity**) will protect a system is a discredited idea. In fact, attackers are very likely to discover anything that has merely been obscured, if it is of enough interest.

The implementation of such security measures cannot be considered in isolation, as there is a trade-off between how many user-oriented security policies are applied and ease of use. In addition, such requirements can add significantly to the cost and administration of a distributed system.

Organisations may also be subject to legal responsibilities, such as data protection regulations, human rights regulations, and freedom of information laws, which may differ from one country to another. All play a part in deciding how much security to apply and to which parts of a system. It may therefore be desirable to apply a local or international standard for information security management, such as the International Organization for Standardization standard ISO/IEC 17799:2005.

SAQ 1

For each of the following scenarios, which type of interference is taking place?

- (a) Alice alters Bob's file without permission.
- (b) A hacker deploys software to observe packets travelling on a network.
- (c) A website receives a flood of requests for a web page, preventing callers from viewing the site.
- (d) A user receives an email appearing to be from his supervisor, who did not send the email.

ANSWER.....

- (a) Modification
- (b) Interception
- (c) Interruption
- (d) Fabrication

Exercise 1

Describe how each of the classes of security threat shown in Figure 1 may apply to a file on a hard disk.

Discussion

- (a) Normal operation – no interference to file access. (Information/operation not threatened.)
- (b) Interruption – an attacker has prevented authorised access to the file.
- (c) Interception – an unauthorised party has obtained access to the file.
- (d) Modification – an unauthorised party has altered the file.
- (e) Fabrication – a fake file has been constructed.

Exercise 2

- (a) What security services might be at fault in a click-fraud case in which fake clicks are generated on an online advertisement?
- (b) What class of threat is this an example of?

Discussion

- (a) Non-repudiation may be at fault if the clicks cannot be associated with particular parties. It is likely that the clicks cannot be easily traced, because the most likely form of identification in this case is an IP address, and IP addresses can be fabricated by a program.
- (b) The class of threat is fabrication. (Fabrication prevention is a reason for some websites setting ‘puzzles’ that need to be solved before a certain action can be carried out – completion of such tasks requires human involvement.)

Exercise 3

The Music Store sells instruments and accessories (reeds, strings, sheet music etc.). Customers have unique account numbers. Some customers pay with cash and some on credit. Goods that are ordered on credit are sent with an invoice to the customer and payment is required later. Not all customers have credit accounts, and those that do have a spending limit. What security issues are involved for The Music Store’s online purchasing system?

Discussion

The main issues are the following.

- ▶ Authentication is needed to establish customer identity.
- ▶ Authorisation is needed for credit purchases and for their amount, also to ensure that callers can only perform functions that they are authorised to execute and access resources that they are allowed to access.
- ▶ There must be a way of proving that a customer made an order (non-repudiation for buyer and seller).
- ▶ Confidentiality of customer information must be maintained.
- ▶ The integrity of the order must be maintained; for example, the values on the invoice data must not change.
- ▶ Availability of The Music Store system is important for business!

3

Cryptography

We now turn our attention to how the contents of communications can be made confidential, so that if they are intercepted the damage is limited. This is the work of the science of **cryptography**. A cryptographic algorithm employs a **cipher**, that is, an encryption algorithm, which is a way of converting **plaintext**, or open communication, into **ciphertext**. The algorithm changes the plaintext in some way to make it difficult to interpret. The reverse process is called **decryption** or **deciphering**.

We start with a discussion of basic ciphers for illustration and give an idea of how strong they are in terms of how easily an encrypted message can be deciphered.

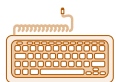
We aim to give you a feel for the classes of cipher used in enterprise systems, when various kinds of cryptography are used in large systems, and why they are appropriate in those contexts. However, we will not require you to write complex ciphers yourself, as you have the Java Cryptography Architecture at your disposal and this provides you with implementations of standard algorithms and techniques.

3.1 Java Cryptography Architecture

Writing strong ciphers is not a task to be attempted by non-specialists. For this reason, enterprise systems must provide built-in cryptographic services in such a way that programmers need not understand the details of the operation of each system, nor to code parts of algorithms.



Java security overview.



Activity 10.1

Cryptography services.

Sun Microsystems defines its Java Cryptography Architecture (JCA) as a 'framework for accessing and developing cryptographic functionality for the Java platform'. The architecture includes the Java Cryptography Extension (JCE), which provides support for various ciphers. In the language of the architecture, there are various **cryptographic services** and they may come from various **providers**.

As this framework is provided in the Java language, it is portable and extensible. There are a number of interfaces that providers of cryptographic services can implement and abstract classes that may be extended by providers to customise services.

Providers can be certified according to external standards, and organisations can choose to subscribe to those standards or not, as the case may be.

For example, code in military applications may require the provider to have passed some standard.

Because there can be various providers, the architecture is implementation and vendor independent wherever possible. The advantages of this are the following.

- ▶ A program can request a particular type of object, for example a `Signature`, and either accept the default provider or specify the provider.
- ▶ The programmer need not worry about the algorithms employed.
- ▶ A provider can update a service without the program needing a change.
- ▶ The use of types such as `Signature` also means that different implementations of the service are interoperable.

Sun Microsystems provides implementations of many standard cryptographic algorithms within the core Java distribution, and these are also available within Java EE systems.

3.2 Ciphers

We start with a simple example of encryption, to demonstrate the process.

The Caesar cipher

The simplest encryption schemes involve replacing each alphabet character with another, known as **substitution**. This is the technique used by the Caesar cipher, reputed to have been used by the Roman leader Julius Caesar to communicate with his generals. In this scheme, a character is replaced by another at a fixed offset along the alphabet. For example, with an offset of 1, an A becomes a B, a B becomes a C and so on (a Z becomes an A). A scheme such as this is expressed as a mathematical function very easily.

The offset value can be considered to be a **key** to the code, that is, a value that allows you to perform encryption or decryption. Reversing the code can easily be done if we know the key.

SAQ 2

How many possible Caesar ciphers are there in English?

ANSWER.....

Assuming a 26-character alphabet (ignoring, for example, punctuation), there are 25 possible substitutions.

Encryption

The main step of this cipher can be expressed using modular arithmetic. For example, if A is 0 and Z is 25, in a 26-character alphabet, the Caesar cipher for a single character can be expressed in pseudocode as

$$c = (p + k) \bmod 26$$

where *c* is the ciphertext for the plaintext character *p*, and *k* is the key.

For example, when *k* is 1, the word FEZ (5, 4, 25) encrypts to GFA (6, 5, 0).

Decryption

Decryption can be performed using the conversion

$$p = (c + k') \bmod 26$$

where *k'* is the decryption key and has the value 26−*k*. In our example, this means calculating

$$p = (c + 25) \bmod 26$$

For example, G will map to (6 + 25) mod 26, which is 31 mod 26, which evaluates to 5, which is F. Likewise, A maps to (0 + 25) mod 26 or 25, which is Z. Therefore, we can retrieve the original word FEZ from the encoded text with as little effort as when it was encoded.

The process of encryption and decryption is illustrated in Figure 2.

The *mod* (modulo) operator works the same way as the % operator in Java, producing the remainder after division (that is, 5 mod 3 evaluates to 2).

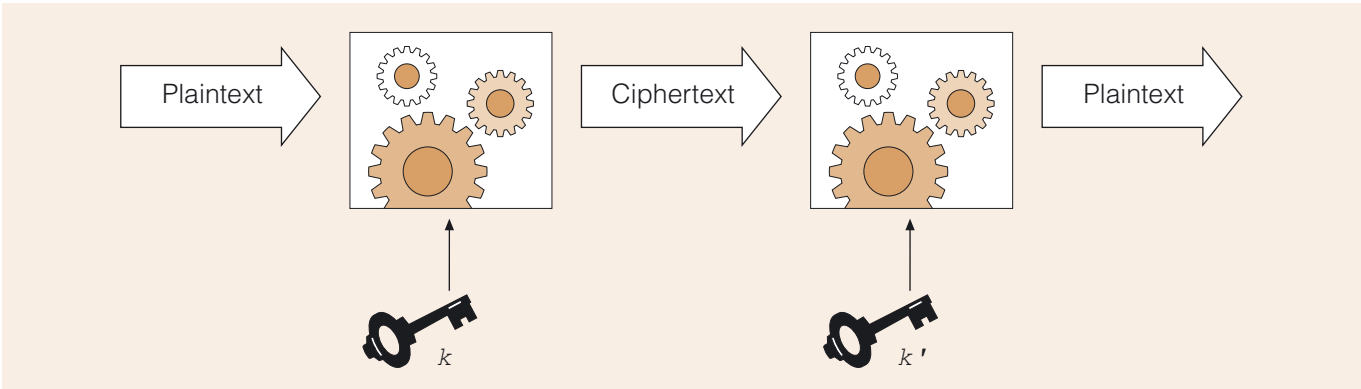


Figure 2 Conventional encryption

An important point is that a cryptographic approach, such as we have described, actually represents a set of such possible transformations, with a set of valid keys. In other words, the same algorithm can be deployed again and again, with different values for the key.

The Caesar cipher is easy to implement, which is an advantage, but it will not stand up to scrutiny from someone with a real interest in discovering the contents of an encrypted message. If we know that this code is being used, a **brute-force** approach, in which all possible keys are tested, would require only 25 decodings, one of which would be correct. The set of possible keys in a code is known as the **key space** – the Caesar cipher can be said to have a small key space since the encryption can be cracked by trying a small number of keys.

In fact, we do not even have to know that a Caesar cipher is being employed, to crack this kind of cipher. For any code in which a single cipher character stands for a single plaintext character, **statistical analysis** of character frequency will crack the code, given enough ciphertext, since in any language characters appear with predictable frequencies.



Letter frequencies.

In practice, a codebreaker would also use any other information at their disposal, including knowledge of the sender, ideas as to the content of the message, or repeated information across a number of messages, such as a date or greeting. We call these techniques **cryptanalysis**.

The Caesar cipher is too weak for serious use, but it has been used in online newsgroups to obscure parts of postings, for example, punch-lines of jokes, or material likely to offend. Many newsreader client programs have built-in functions to decode or encode text in this fashion using what is known as ROT13 substitution, in which letters are shifted by 13 characters.

So, what do we require to produce a stronger cipher?

The rail-fence cipher

One observation on the weakness of the substitution cipher is that the order of the letters in the plaintext has been preserved in the ciphertext. If the relationship between the two is made more complex, the message should be harder to decode. This leads to the idea of a **transposition** code, which results in a jumbling of the letters in the ciphertext.

In the rail-fence cipher, the plaintext is first written as a series of diagonal lines over a number of rows. The diagonals are the 'rails' and the rows are the 'fences'. For example, if we have three fences and seven rails, the message 'FEZ MAN RETURNS FRIDAY' is written as in Table 1.

Table 1 Rail-fence cipher construction

F	M	R	U	S	I	Y	
	E	A	E	R	F	D	
		Z	N	T	N	R	A

The encrypted text is now read off across the rows:

FMRUSIYEAFERFDZNTNRA

In effect, this is an anagram of the original text. This is not as strong as it might look – you can read the original letters at intervals of seven letters. We could make this stronger by combining it with a character substitution, but this would not alter the frequency of the characters in the ciphertext, meaning that statistical attacks are still possible.

However, a combination of the techniques of transposition and substitution can lead to stronger codes, with the aid of a **polyalphabetic substitution**, in which a single plaintext character may be replaced by different ciphertext characters at different times, so obscuring the statistical pattern in the code. For example, a Caesar cipher could be applied in which the key changes every 26 characters. The first 26 characters could be encoded using a key of 1, the next with a key of 2 and so on until the keys must be reused.

Information theorist Claude Shannon (1949) formalised these ideas as 'confusion' and 'diffusion'.

Enigma

A mechanical device for performing a combination of transposition and polyalphabetic substitution is the rotor machine, of which the famous German Enigma machine of the Second World War is an example. The allies captured an Enigma machine, but still had to work hard to break the code. The whole story is a long and interesting one, which we do not have space to explore here. Interested readers may read more online, for example, at the Bletchley Park website.



Bletchley Park.

Block ciphers

Another way of obscuring statistical properties in ciphertext is to encipher text in units (known as blocks) of more than a single character. If the block size is large enough, and an arbitrary but reversible substitution is applied, then the statistical properties of the language are hidden.

For example, if the block size is two characters, there are 26 × 26 possible blocks:

AA, AB, AC, AD, ..., AZ,

BA, BB, BC, BD, ..., BZ,

and so on until we reach ZZ.

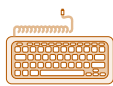
Now we progress through our message and encode any such block by a substitution. For example, ABACUS is broken into AB, AC, US and each of these blocks maps to another pair of characters according to some key. Because we are encoding blocks of two characters at a time, statistical properties of occurrences of individual characters do not help in cryptanalysis, though frequency of occurrences of pairs of characters do! In this case, the cipher is not a lot stronger than a Caesar cipher, because digraphs (that is, pairs of characters) occur with predictable frequency in long texts. However, as the block size grows, statistical properties of the language are obscured.

Decoding such a cipher requires knowledge only of the key used to encode the plaintext. The mapping can then be reversed to reproduce the original text.

This gives rise to the idea of a **block cipher**, which transforms fixed-length blocks of plaintext into ciphertext. Messages need to be broken into blocks of the right length, or padded, where necessary, to make up full blocks. Such ciphers have what are termed ‘modes of operation’, that is, algorithms for dealing with long texts. For example, each block may be encoded independently (such as in the manner we have just suggested for a block size of two characters), or the encoding of each block may depend on other factors, such as the values of the blocks adjacent to it.

Most modern encryption ciphers are block ciphers. The strength of these schemes depends on their mode of operation, the block size and the complexity of the algorithm employed.

The first widely used encryption standard was the block cipher known as the Digital Encryption Standard (DES), established in 1976. Since 2002, DES has been largely superseded by the Advanced Encryption Standard (AES) and Triple DES, which applies DES three times according to a more elaborate scheme. We will not be discussing the details of how these or other industry standard algorithms work.



Activity 10.2
Symmetric encryption.

SAQ 3

How would you decode an English message encoded in ROT13, assuming a 26-character alphabet?

ANSWER.....

You would apply the same function again! The cipher is its own inverse. Two shifts of 13 return you to your starting point in the English alphabet. In this case the encryption key and the decryption key have the same value.

Exercise 4

What advantages does a cipher involving modulo arithmetic have?

Discussion

The output of a modulus function is always within a certain range, which can be easily mapped to the desired character set. It is also easily implemented and computationally cheap.

3.3 Strength of ciphers

How can we characterise the strength of our codes? One way is in terms of the number of guesses you would have to make to find the key, using brute force. In this case, on average, you would need to test half the possible keys to find the correct one. At worst, you would need to try them all. We will consider only the worst case in this discussion.

Given an amount of time to perform each test, you could say that the worst-case time to crack the code $T(n)$ is related to the size of the key space (the number of keys in the key space) by the mathematical function

$$T(n) = kn,$$
(1)

where n is the size of the key space and k is the amount of time to do one test.

Equation 1 tells us that each time we add 1 to the size of the key space, the value of $T(n)$ grows by k . The proof of this is simple. We can find the difference between the time required for a key space of n and one of size $n + 1$:

$$\begin{aligned} T(n + 1) - T(n) &= k(n + 1) - k(n) \\ &= kn + k - kn \\ &= k. \end{aligned}$$

We can also relate the worst-case time to crack the code to the length of the key employed, measured in bits, on the basis that the keys are numbered from 0 upwards. For example, if the key is represented by two bits, there are four possible keys, represented by the binary numbers 00, 01, 10 and 11. At worst, a codebreaker would require four guesses to find the correct key. Likewise, if there are three bits, there are eight possible keys. If there are b bits, the number of possible keys is 2^b . This means that the relationship between the worst-case time to crack the code and the number of keys can also be written as

$$T(b) = k2^b, \tag{2}$$

where b is the number of bits in the key and k is the amount of time to do one test.

Equation 2 tells us that each time one extra bit is used, the key space *doubles* so the work required *doubles*. For example, the work required to check eight keys ($b = 3$) is twice as much as the work required to check four keys ($b = 2$). Again, it is simple to show this more generally by looking at the ratio of the time to crack the code for $b + 1$ bits versus the time to crack the code for b bits:

$$\begin{aligned} T(b + 1)/T(b) &= k(2^{b+1})/k(2^b) \\ &= 2^{b+1}/2^b \\ &= 2^{b+1-b} \\ &= 2^1 \\ &= 2. \end{aligned}$$

Mathematically, we say that the amount of work required to crack the code is **linear** with respect to the key space size (as shown in Equation 1), and **exponential** with respect to the number of bits in the key (as shown in Equation 2).

Consider the following table of values of b and $T(b)$, in which we assume, for the sake of argument, that k , the amount of work required, has the value 1. The value of k could represent 1 second to do the test, for example. The final value in the table is an approximation.

Table 2 Time in seconds to crack a code with b bits in the key

b	1	2	3	4	5	10	20	100
$T(b)$	2	4	8	16	32	1024	1 048 576	$1.267\,650\,6 \times 10^{30}$

Notice how quickly the time $T(b)$ grows as the value of b grows.

Our example above illustrates how key space size relates to the difficulty of cracking a code, but usually increasing the key space size also affects the work we have to do to encode or decode text. Thus, there is a trade-off between the time required for us to use long keys and the strength of the code as reflected in the size of the key space. Our aim in choosing a cipher will be to make the key space large enough to resist a brute-force attack, while keeping the computation required to perform encryption and decryption manageable.

Information theorist Claude Shannon likewise took the maxim ‘the enemy knows the system’ as a starting point for discussing cryptography.

In practice, any algorithm employed must obscure plaintext in such a way that the main measure of its strength is not its algorithm, but the length of the key (and hence the size of the key space) it employs. This is the basis of all modern ciphers and puts into practice a principle established by an early commentator on ciphers, Dutch cryptographer Auguste Kerckhoffs, who wrote in the 1880s about military applications of codes. Kerckhoffs set out several principles for practical ciphers, including ease of use and ability to transmit by telegraph. His most famous statement, however, is that it is best to assume that the enemy knows the system employed, and that its security should depend only on secrecy of the key. In other words, knowledge of the encryption algorithm employed should not be an advantage to a codebreaker.

The acceptable length of a key changes over time as technology advances. For example, the DES cipher uses 64-bit keys (of which one bit in each byte is used for a parity check, reducing the effective strength of the key to 56). This was considered strong enough for DES to be classified as munitions by the US government and it was therefore subject to export restrictions. However, in 1999 a grid-computing project succeeded in decoding a DES encoded message, demonstrating that a key length of 56 was no longer sufficient. More recent algorithms rely on longer keys, and a key length of 1024 bits may now be considered necessary for strong encryption.

As key security becomes paramount, key management becomes an important consideration. For example, if a key is reused, cryptanalysis may be aided, so keys often expire after some time, or are one-use only, known as **session keys**. It is also important that there is not a pattern in key use that might allow an attacker to predict future keys.

A concept related to the session key is the use of a **nonce** value, which is a random number used once and then discarded. This can be used in authentication procedures to prevent a replay attack. An example is a server sending a client a nonce value that is then used by the client in generating a password sent in encrypted form. A subsequent attempt by an attacker to authenticate by replaying an intercepted password will fail, because the server will not accept the same value again.

Exercise 5

Based on the values in Table 2, how long does it take to find the right key by brute force when the key lengths are the following?

- (a) 10 bits
- (b) 20 bits
- (c) 100 bits

Discussion

- (a) 1024 seconds, or about 17 minutes.
- (b) 17 476 hours, or about 728 days.
- (c) Too long!

Exercise 6

Which is stronger under a brute-force attack: using a password of five characters in length from the character set {a, b, c, d, e, f} or using a password of ten characters in length from the character set {a, b, c}?

Discussion

If you have six characters to choose from and a password of length five, there are six choices for the first character and six for the next and so on. The key space size is $6 \times 6 \times 6 \times 6 \times 6 = 6^5$ or 7776.

If you have three characters to choose from and a password of length ten, there are three choices for each character. The key space size is $3 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3 = 3^{10}$ or 59 049.

The longer password is stronger, because it results in a larger key space.

However, note that in practice even a long password that appears in a dictionary is relatively weak, because an attacker can perform a dictionary attack, that is, use a dictionary as a source of possible passwords to test.

Exercise 7

If you used brute force, how many keys, on average, would you have to check to decipher a message encoded with DES, given that the effective size of the key is 56 bits?

Discussion

You would have to check half of the 2^{56} possible keys, which is $2^{56}/2 = 2^{55}$. (That’s a lot.)

3.4 Symmetric cryptography

Encryption based on substitution and permutation was the basis of most cryptography systems until the 1970s. In most systems, the encryption key is the same as the decryption key and we call the scheme **symmetric**. Because anyone knowing the key can decipher an intercepted message, the key must be kept secret and the sender has to communicate the key to the receiver in some way (perhaps offline) in order for the receiver to decipher the message. For this reason, the key in these systems is known as a **secret key**.

Let us suppose that Alice wants to send a message to Bob, while Eve is intent on eavesdropping.

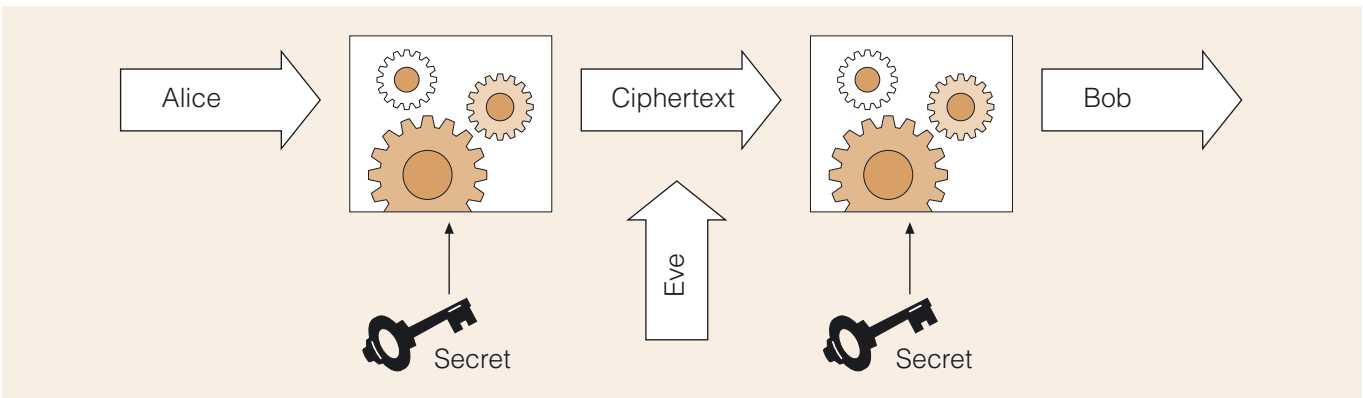


Figure 3 Symmetric key cryptography

Figure 3 shows that Alice can send a message to Bob using a secret key and that Bob can use the same key to decipher the message. In fact, either Alice or Bob in this case can use the secret key to send a message to the other.

Eve, a middle person, may intercept the message, but does not have access to the secret key. The harder it is for Eve to decode the message without knowledge of the key, the stronger we say the encryption is.

Of course, the key is vulnerable to interception when it is being communicated, so some trusted means of doing this is required. Sharing a key would typically take place using a face-to-face meeting or through a trusted third party sending the key to Alice and Bob. It is also possible that Alice or Bob might reuse a recently shared key to encrypt and transmit a new key to the other.

Although the encryption can be made very strong in secret key cryptography, this problem of key distribution remains a weakness, and so key management becomes a significant issue.

3.5 Public key cryptography

The problem of key distribution led to research on ways of securely distributing keys and removing reliance on the need to share a secret key, which is one of the great weaknesses of conventional encryption. This led to the emergence of **public key cryptography**, in which there are two related keys, one of which can be made public without compromising the encryption. This is also known as **asymmetric cryptography**: the keys for coding and decoding are not the same.

History shows that this approach was recognised separately by different parties, but the first published account is due to Whitfield Diffie and Martin Hellman in 1976 in their now famous paper titled ‘New directions in cryptography’.

How might such a system work in practice?

Exponentiation cipher

The operation of an asymmetric cipher may be illustrated as follows. Consider a ciphering function whose value depends on raising the numeric value of each plaintext character p to some power:

$$c = p^k \bmod n,$$

where k is the key.

In other words, the value for the input is multiplied by itself a number of times equal to the key value, and then a remainder is taken to find the ciphertext output value c .

In this cipher, the values of p , n and k must be chosen to ensure that each input ciphers to a unique output, otherwise there will not be a unique decoding of a ciphertext. For example, using input values of 0 or 1 is not desirable here, as raising these numbers to a power does not alter their value. Therefore, we will have to number our alphabet from 2.

Suppose that we have a small alphabet (which allows us to produce a more workable example) with just the letters A, B, C, D, E, F, G and H. Let us take as an example a k value of 3 and a modulus n of 11. The cipher values in Table 3 are produced.

A number of modern encryption schemes rely on exponentiation functions.

Table 3 Exponentiation cipher, $k = 3, n = 11$

p	A	B	C	D	E	F	G	H
p value	2	3	4	5	6	7	8	9
p^3	8	27	64	125	216	343	512	729
$p^3 \bmod 11$	8	5	9	4	7	2	6	3
c	G	D	H	C	F	A	E	B

Although this amounts to a simple substitution of one letter for another, we have used a simple example – in practice the algorithm would be more complex, but still reversible.

The decryption function works in the same way as the encryption function, but with a different key:

$$p = c^{k'} \bmod n,$$

where k' is the decryption key.

In this case the decryption key is not related to the encryption key by a simple relationship as was seen in the Caesar cipher, and not all choices of k and n produce reversible mappings. For some choices, two or more letters may map to the same output. However, it is possible to describe the properties of suitable values of k and n so that a unique decryption key can be guaranteed.

It happens that the following conditions are required for reversibility in our example.

- ▶ The value of n should be prime and larger than the largest plaintext code value.
- ▶ The value of k should not have any factors in common with the value $n - 1$.

This ensures that there is a unique decryption key k' satisfying a relationship

$$k \times k' \bmod (n - 1) = 1.$$

In our example, with $k = 3$ and $n = 11$, the value of $k' = 7$ satisfies this, because

$$3 \times 7 \bmod 10 = 21 \bmod 10 = 1.$$

These calculations are difficult by hand, especially when there is a larger alphabet, and higher values of k and n are used; however, there are quick computer algorithms to do the work.

Notice that, because of the relationship between the keys, the values of k and k' are interchangeable: either could be used for encryption and the other will perform the decryption. This is a common feature of such algorithms. In practice, one of these values is kept secret and called the **private key**; the other is passed on to a message sender and is known as the **public key**.

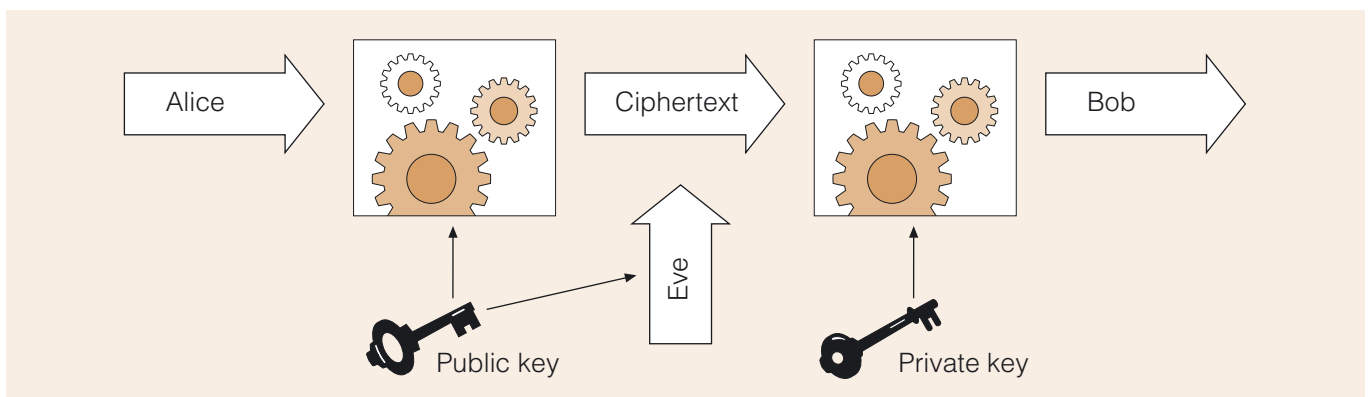


Figure 4 Asymmetric key cryptography

Figure 4 illustrates the way in which this system can be put to work to provide confidentiality. Alice encrypts a plaintext using Bob's public key. Bob receives the message and deciphers it using his private key.

Eve can see the public key, but deciphering the message requires knowledge of the private key.



Activity 10.3

Asymmetric encryption.

Strength of asymmetric cryptography

Suppose that Eve is lucky enough to intercept a plaintext message and its encryption, and that she knows that the exponentiation cipher we have just described is being employed. How would she find the value of k' that makes the following equation true?

$$p = c^{k'} \bmod n$$

In order to do this, Eve would need to reverse the exponentiation, known as a discrete logarithm calculation. It so happens that, when combined with a large modulus calculation, this is a difficult thing to do. With the best algorithms we know of, the time to perform this calculation grows exponentially with respect to the number of bits in n .

It can also be shown that the number of suitable key pairs in this cipher is about 2^{b-1} , where b is the number of bits in n , so that for large values of b the key space is large and a brute-force search will be impractical.

Functions that exhibit these properties – easy to perform in one direction but hard to reverse – are known as **one-way** functions. A special type of one-way function, in which knowledge of a secret key enables the reversal of the function, is called a **trapdoor** function. Both kinds of function are extensively used in public key cryptography.

A second standard one-way function in use in current encryption methods depends on the fact that it is easy to multiply two large prime numbers together to find their product, but it is hard to find the two prime factors of that large number. With the best known algorithms, the computation required to factor the encryption key into primes to find the decryption key grows exponentially with respect to the number of bits in the key.

Thus, the strength of public key encryption relies on the computational difficulty of inverting the encryption function to determine the decryption key and on the infeasibility of a brute-force search on the key space.

Although the example we have given is too simple for practical use, it helps to illustrate the principles underpinning these systems, which are as follows.

- ▶ Two keys are used: one to cipher and the other to decipher.
- ▶ The generation of keys must be computationally easy.
- ▶ Encryption and decryption using the appropriate keys must be computationally easy.
- ▶ It is not computationally feasible to decipher a message given the public key, even if you know the algorithm.

Increasing the key length also increases the amount of time required to perform the encryption. However, the key length will be chosen so that the amount of work to perform the encryption will be kept at a manageable level, while the computation required to decipher the code is unmanageable. It is therefore possible to create messages that are, in practice, secure, because cracking the code is too much work.

With public key cryptography, if you need to send a message to multiple recipients, the encryption process needs to take place once for each recipient, since each recipient will have a different private and public key pair. This can be a consideration if the encryption process is computationally expensive.

An example of a public key system is RSA, which was designed by Ron Rivest, Adi Shamir and Len Adleman at MIT and takes its name from the first letters of its creators' surnames. RSA is an exponentiation-based algorithm whose strength relies on the assumed difficulty of factoring large numbers.

Digital keys, like physical keys, must be handled carefully in order for systems to be secure. Since the cryptographic systems themselves are relatively secure, access to the keys themselves may become the primary target of an attacker. If a private key is lost for some reason, the system breaks down and a new key pair must be generated.

SAQ 4

Why is public key cryptography called asymmetric and private key cryptography called symmetric?

ANSWER.....

In public key cryptography there are two different keys, the public key and private key, one used for encryption and the other for decryption. In private key cryptography there is one key, the secret key suitable for both encryption and decryption.

Exercise 8

Suppose that you intercept a ciphertext message consisting of the characters AB. Using the decryption function $p = c^{k'} \bmod 11$ and the decryption key 7, find the original text. Assume that the alphabet is numbered from 2.

Discussion

The decryption key is 7. The ciphertext is AB. For the cipher value A, we have $p = 2^7 \bmod 11 = 128 \bmod 11 = 7$, which implies that the input value was F. For the cipher value B we have $p = 3^7 \bmod 11 = 2187 \bmod 11 = 9$, which implies that the input value was H. Hence the plaintext was FH.

Exercise 9

Give an example of a middle-person attack on public key cryptography.

Discussion

Alice wants to send a message to Bob, using Bob's public key. Alice is tricked into using Eve's (the middle person's) public key instead. Now Eve is able to impersonate Bob and read messages intended for Bob. (Bob will not be able to decipher these messages, even if he receives them.)

3.6 Authentication

Another issue in secure communication is how a recipient of a message can tell who or where the message came from, known as authentication. This is necessary to avoid impersonation and also to assist in establishing accountability of the sender (non-repudiation).

Historically, authentication measures have relied on the production of something that is impossible or very difficult to copy, and on degrees of trust between the communicating parties.

- ▶ A trusted messenger could carry the communication.
- ▶ Handwritten signatures have long been used for authentication, but may be forged. They also change through time (as the individual's handwriting changes) and are difficult to verify accurately.
- ▶ Signet rings that could be impressed into hot wax have been used to seal envelopes, to prove that they have not been tampered with. The impression left by the ring (also called a seal) would break if the envelope were opened.
- ▶ In recent years, we have all been required to remember PINs (personal identification numbers). This approach may give way to alphabetic passwords (as commonly seen in computer systems already) or biometric identification, such as fingerprint or iris recognition.
- ▶ A **challenge** is also a general approach to authentication, such as asking a telephone caller for a postcode or their mother's maiden name.

There are other factors to consider. Any widely deployed authentication measure needs to be acceptable to the users, while usability and cost also need to be taken into account. These systems may also provide greater or lesser degrees of non-repudiation.

The problem faced in computer systems is roughly the same as Alice appearing at a shop with a debit card and authenticating using a PIN. Shops accepting her card do so because they trust that the bank has authenticated Alice and ensured that she, and not Eve, received the card and that only she knows the PIN. The shop accepts any card from BankoClaro, because they trust that bank. Cards from ClaroBanko they do not trust, because they have never heard of it. In this way, Alice is authenticated to a shop.

This is a **hierarchy of trust**, with the bank being trusted, and being trusted to have authenticated the card bearer, therefore allowing the shop to put trust in the bearer.

How do we introduce these elements in digital form?

Digital signing

Public key algorithms can perform the extra function of authentication if the keys are interchangeable. In this case, Bob can send a message to Alice encrypted with his private key, and it will decode using the public key. While this does not provide confidentiality (Eve can also read the message using the public key), it is a form of proof that the message originated with Bob; in other words, it is a form of authentication.

We call such a scheme used to verify the origin of a message a **digital signature** and we say that the message is **digitally signed**. As confidentiality is secondary in this case, rather than encrypting the whole message, which may be inefficient, we may use a **message digest**, a reduced-size version of the message.

A message digest can be created using a **hash function**, which is simply a mathematical function that maps an input to a small output. (We will return to hash functions in Subsection 3.7.)

Figure 5 shows that a hash function can be used on a message to generate a digest. The digest can then be encrypted using a private key and appended to the message as a digital signature. The resulting message can then be sent to Alice.

If the bearer gives the card to somebody else, and tells that person the PIN, he or she is extending the hierarchy.

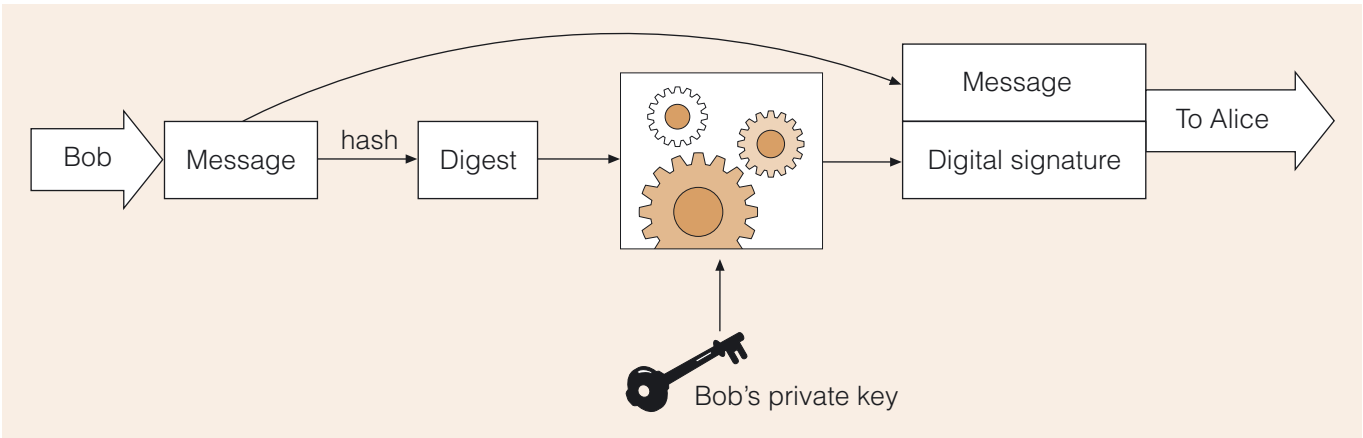


Figure 5 Digital signing from Bob's point of view

Now when Alice receives the message, she can use the same hash algorithm (which they have agreed earlier) to generate the digest again, and use Bob's public key to decrypt the digital signature. The two should have the same value.

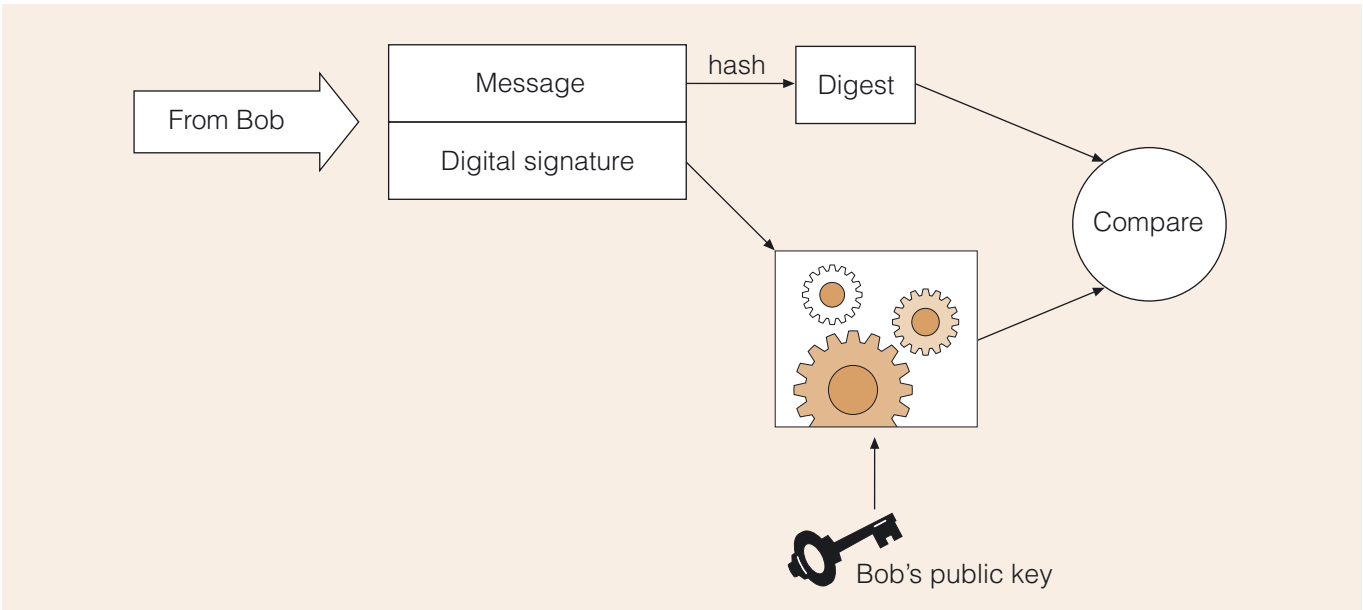


Figure 6 Digital signing from Alice's point of view

Figure 6 illustrates how Alice uses the signed message. She regenerates the digest from the message. The digital signature attached to Bob's message is decrypted using Bob's public key. Alice then compares her digest with the decrypted signature. If the message was signed with Bob's private key, then these should have the same value.

Exercise 10

Does Alice really know that a message deciphered by Bob's public key came from Bob?

Discussion

This depends on how Alice received the key. If she did not meet Bob in person or received the key in an insecure manner, she knows only that the public key comes from someone claiming to be Bob.

Certificates

Exercise 10 illustrates that a problem remains. Anybody could have posted a public key online and claimed to be Bob. Alice may not fully trust a received message, and furthermore Bob may be able to claim that he did not send the message. To provide non-repudiation and trust in the origin of a signed message, we require a way of associating Bob with his keys. We call this a **digital certificate**.

Bob's certificate would contain the public key matching his private key, but now a **certifying authority** will be trusted to ensure that the certificate does actually come from Bob. If the recipient of a message can verify that the certificate originated from the certifying authority and trusts the certifying authority to have done its job, then the contents of the message can be trusted to have come from Bob.

A little thought raises the question of why we should trust the certifying authority any more than we should trust a claim that a published key is Bob's. In fact, it is necessary for certifying authorities to be certified in turn, and they receive so-called **root certificates**, issued by a root certifying authority, which is an organisation or party that is trusted by the communicating parties. Root authorities have a similar status to trusted banks in our debit card example.

The root authority can issue different kinds of digital certificate according to the purpose and level of checking performed. For example, VeriSign is an authority that issues:

- 1 server certificates (for web servers and secure communication with web browsers);
- 2 developer certificates (to sign software that will be made available for download);
- 3 personal certificates (for communication between individuals).

These certificates are issued at three different classes for which the certifying authority provides different levels of checking and so different levels of assurance of the holder's identity.

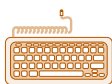
Typically, a certificate may contain:

- ▶ the certifying authority's name;
- ▶ the holder's name (a person's or an organisation's);
- ▶ some contact information;
- ▶ the class or type of certificate;
- ▶ a start date and an expiry date;
- ▶ the holder's public key, to decrypt the certificate.

In order to prove that the certificate comes from the certifying authority, it must itself be signed, employing the certifying authority's private key so that its origin can be verified using the certifying authority's public key.

Many web browsers and mail readers support certificates and will automatically unpack them so that the recipient can verify the contents of the certificate. The recipient need not know that encryption and decryption has taken place. The software will simply unpack the certificate using its attached public key and automatically contact the certifying authority to verify that the certificate was correctly issued and is still valid.

A certifying authority associates users with their public keys using a **public key infrastructure (PKI)**. A PKI may include protocols, management systems, communications standards and encryption methods, for example. Such systems will include ways of registering keys, storing them and expiring, revoking and destroying them. Not only does this provide proof that the certificate holder is legitimate, but the public key attached to the certificate can also be used to send secure communications to the certificate holder, who has the private key required to decipher the message.



Activity 10.4

Digital signing.

A less hierarchical approach is to build a **web of trust**. In this approach, a group can establish a set of trusted certificate holders. When a new certificate is submitted, existing users can endorse it. Other users can decide whether to accept the new certificate, based on the trustworthiness of the endorsers. This flexible approach has been used by Pretty Good Privacy (PGP), an open-source public key cryptography and authentication system invented by Phil Zimmerman and used widely in email for confidentiality.



Pretty Good Privacy.

Of course, just because a program or communication is digitally signed does not make it safe. A program could contain errors or the communication could be false.

Exercise 11

State three reasons why keys might have expiry dates.

Discussion

- 1 Reusing a key may give an attacker more data to work with and so a better chance of cracking the code.
- 2 Keys that previously were considered secure might not be any more due to increased computing power and new methods of attack.
- 3 If a key has become compromised, it cannot be used indefinitely.

3.7 Hash functions

When discussing digital signatures we met hash functions, but we have not yet described their operation or more general application in security systems.

A typical hash function provides a quick way of associating an input with a numerical output and can map from a large input space onto a relatively small output space. The standard application is to create a table (hence the terminology ‘hash table’). This allows a user to store and retrieve data very quickly, because computing the index into the table is typically much faster than searching through a table for a value.

Whereas mathematical functions may be reversible (you may be able to work out what the input must have been, given the output), hash functions are not reversible.

A **cryptographic hash function**, such as is used in creating a message digest, is a one-way function and typically will operate on strings.

A simple example is a hash function for strings that sums the ASCII values of the characters in the input string to produce its output. Although this does not lead to a unique output for every string, it provides a quick way of associating the input with an index into a table. When two inputs map to the same output, we say there is a **collision**. There are various schemes for resolving this, for example:

- ▶ incrementing the index until a space in the table is found;
- ▶ building a list of entries at each index;
- ▶ combining more than one hash function output so that there is added variability in output values.

If collisions are common, an attacker may fabricate a text or alter an intercepted text in such a way that it computes an identical hash, so there is a danger of substitution of a fake message for a real one.

The strength of a hash can be quantified according to the likelihood of a collision and according to how much computational effort has to be expended in order to identify such an occurrence. These are related quantities – if the probability of collision is high,

then a brute-force approach would quickly identify a fake message with the same hash value. This probability also relates to the randomness of the output value that a hash function produces.

Uses of cryptographic hash functions are as follows.

1 Confidential storage or transmission of sensitive information

Systems do not typically store or transmit plaintext passwords, but rather use encrypted or hashed passwords. With a hash function, there is no key to be lost and, if the password file is compromised, it is still hard to reverse the encryption.

Because users may supply the same password, the system may add **salt** (that is, some 'noise' or perturbation) before hashing, so that identical input values do not map to the same hashed value. The system stores the salt value and the hashed value, and can reuse the salt value later to recompute the correct hashed value. This makes dictionary-based attacks more difficult and also, should an attacker gain access to the password file, helps to obscure cases when users have picked the same password. (Failing to do this can mean that an attacker can determine when users have used the same password, or attempt to hash a password until the hashed value matches one in the password file.)

2 Time stamping

In networked systems, communications may arrive in a different order to that in which they were sent, and it may be necessary to establish their correct order. One way of achieving this is to use a third-party **time stamping** service. To avoid revealing the contents of communications to the third party, a hash function can be used to generate a digest that can be time stamped.

This technique can also be used in detecting replayed or delayed messages.

3 Checking the integrity of a communication

For example, we could check that the hashed value, also known as a **checksum**, of a file has not changed. In order for this to succeed, we require that a small change in the file will lead to a change in the hashed value, so that an alteration can be detected. (In practice, according to the strength of the function employed, the output is highly likely to change, rather than certain to.)

A checksum can be computed at the source of a communication (it may be based on all, or just part, of the communication), and is sent along with the communication. Then, on receipt, the checksum is recomputed with the same hashing algorithm, based on the received communication. The checksum values at origin and on receipt must match, otherwise it is very likely that the message has been altered in transit.

4 To generate a digital signature for authentication from a message digest

This is the application we discussed in Subsection 3.6 under the heading 'Digital signing', and is used particularly when confidentiality is not important. If the digest is generated from the whole message, integrity may also be checked. This is the case with a **message authentication code (MAC)**.

Secure Hashing Algorithm-1 (SHA-1) produced by the United States National Security Agency is an example of a cryptographic hash function that can be used to authenticate a sender and check the integrity of a communication, based on a shared secret key.

Salt is a general technique to add variability to output.

Exercise 12

- (a) Give an example of how an attacker might exploit a collision.
- (b) Suppose that you intercept the following messages between a user and their bank, in which the login details are encrypted text.

Username: poexmcs
Password: krisjleeer

Could you log in to the banking system? If so, how?

Discussion

- (a) The message ‘Transfer £5 to City Bank’ might have the same hash as ‘Transfer £1000 to City Bank’. The attacker might be able to intercept and substitute the second message for the first.

Likewise, a program could be altered in such a way that its hash is unaltered, so that an integrity check does not detect tampering, and a malicious program can be substituted for a valid one.
- (b) If you could access the system below the normal user interface, at the point at which encrypted information is received, it might be possible to replay the login and password information and impersonate the user. However, this attack is defeated by the use of nonce values.

3.8 Stream ciphers

A different kind of cipher was inspired by a need to encrypt communications of unknown length ‘on the fly’ by telegraph. In 1917, US inventor Gilbert Vernam patented the idea of such a cipher. He proposed that a key at least as long as the message is combined with the plaintext using an XOR (exclusive OR). By recombining the ciphertext with the same key and again applying an XOR at the receiving end, the original text can be recovered. In this case, the key must be shared by both sides before communication can take place, just as with a symmetric cipher, but there is no need to break text up into blocks or to know the length of the communication.

Recall that $A \text{ XOR } B$ works as in Table 4. The XOR is true when exactly one of its operands is true (represented by the value 1).

Table 4 XOR operation

	A = 0	A = 1
B = 0	0	1
B = 1	1	0

An example is shown in Table 5, where a message is sent as a stream of bits and this is combined with a random key.

Table 5 Combination with key

Message	0	1	0	0	1	1	0	0	0	1	1	1	...
Key stream	1	0	1	0	0	1	0	0	1	1	1	0	...
Cipher	1	1	1	0	1	0	0	0	1	0	0	1	...

Notice especially that if the ciphertext is recombined with the same key, it produces the original message. Thus, the receiving side needs only to perform the same operation again, using the same key.

Vernam's idea also led to the idea of a **one-time pad**, which takes its name from the idea of a pad of paper with each sheet having a key on it. As each key is used, the top sheet of the pad is removed and discarded. The receiver of the message has an identical pad and so both parties can keep their keys in step with one another.

American information theorist Claude Shannon showed that, under certain conditions, this cipher has what he termed **perfect secrecy**. This means that the ciphertext does not reveal any information about the plaintext and effectively makes the one-time pad unbreakable. Technically, the probability of cracking the code if you are in possession of the ciphertext is the same as doing so without the ciphertext. However, this finding depends on the keys used being random and unpredictable from one run to the next. Also, the parties must securely share these long keys. These are difficult requirements and so the one-time pad is not frequently used in practice.

Modern **stream ciphers** approximate the ideal of the Vernam cipher by generating a sequence of keys called a **key stream** and combining this stream with the plaintext. The key may vary independently of the data being encrypted as a result of an initial 'seed', or it may be changed according to the data by some algorithm. The key may be changed for every bit of the plaintext, and is combined with the plaintext by a simple and fast logical operation, such as an XOR.

Because the blocks of data transmitted in wireless systems are of unpredictable length, stream ciphers are often used in this context in preference to block ciphers, which would be likely to suffer from a frequent requirement to pad short blocks to the required length. Stream ciphers are also much less complex than block ciphers and so computationally cheaper.

An example of a modern stream cipher is the Rivest Cipher 4 (RC4), invented by MIT's Ron Rivest, which makes use of a pseudo-random stream of key values that depends on an initial shared secret key.

It is well known that the implementation of RC4 in the Wireless Equivalency Protocol (WEP), used in some wireless networks, is not secure. One issue is that the length of the stream is limited and restarts after it is exhausted, allowing an eavesdropper to see two texts encrypted with the same cipher. This opens the encryption to statistical attack.

The same RC4 algorithm is used to better effect in Wi-Fi Protected Access (WPA), a more secure wireless protocol, to ensure that every packet is sent with a unique key.

3.9 Choice of system

The context in which encryption is required will determine the choice of the encryption system. In addition, the strength and the speed of the encryption method are two key points affecting the choice of algorithm.

Secret key systems are typically block ciphers and are preferable:

- ▶ in environments where the keys can be shared securely, for example, they may be used by a bank for internal storage of sensitive data;
- ▶ when there is no requirement to share or distribute the secret key, for example, to securely store data in a local file used by an application.

Public key systems are typically used where keys have to be transmitted by potentially insecure routes, such as via a public network.

Quantum cryptography may also offer unbreakable encryption, but is currently of far less general use.

A seed is an initial value that starts the process of generating keys.

The RC4 algorithm was initially kept secret but was leaked to the Web and has subsequently been deployed under other names including ARCFOUR (known as alleged RC4, to avoid trademark issues).

We have also noted that stream ciphers are deployed in wireless communications, while hash functions are typically used where the original data does not need to be recovered, and for authentication.

Hybrid systems are also used, partly because public key algorithms tend to be computationally much more expensive than secret key ones. For example, a public key system can be used to distribute keys for a symmetric system. The one-off cost of the key distribution is offset by the lower symmetric encryption cost. This approach is followed by the Diffie–Hellman key exchange protocol and by the popular PGP system.

In general, encryption schemes depend on empirical reasoning about their security, and the longer an encryption method is unbroken, the stronger the evidence of its worth. In some cases, research exposes weaknesses, or technology advances to the point where problems thought too computationally expensive can now be computed. However, the problems (such as factoring large numbers) on which many public key schemes are based, are not completely understood. We believe that they will remain hard to solve using current technology, but we have no proof of this.

In distributed systems we settle for security that is effectively good enough, convenient, and computationally easy to perform and hard to break. What this means changes over time as computers get faster, mathematics advances and technology changes, so today's 'good enough' may be tomorrow's insecure.

SAQ 5

What classes of cipher might you be likely to use in the following circumstances?

- (a) Storing passwords in a local file
- (b) Proving you sent an email
- (c) Using a wireless connection on your laptop
- (d) Encrypting files on a file system

ANSWER.....

- (a) Hash
 - (b) Public key and hash (for signing)
 - (c) Stream
 - (d) Block cipher
-

4

Protocols and networking

In this section, we consider security issues related to communication between parts of systems over networks.

It is insufficient for security to be implemented only within components of a distributed system. We also require security when data leaves components and travels across networks to other components of the system. For our purposes, the most important scenario in which this takes place is between a server and a remote client, as commonly seen in ecommerce.

We consider some security implementation choices that may be made in this context, as well as two examples of secure protocols.

4.1 General issues

It is now common for computers from different organisations to be linked. Peripheral and mobile devices may also be networked. However, many of the commonly used protocols in these contexts originated in academic circles and the issues raised by present day internet use could not have been foreseen. As a result, exploitation of weaknesses in protocols is the basis of many attacks on distributed systems.

As in other contexts, both passive and active attacks are possible in network communications; that is, they may take the form of eavesdropping (interception) or they may alter data in some way. Whilst it is not a panacea, the simplest and in many ways the most robust way in which an organisation can secure its communications over a number of sites is to run its own wide area networks through leased lines, which may also provide greater reliability, at the cost of higher maintenance.

If this is not practical or desirable, the usual approach is to use cryptography as a basis for secure communication protocols. Newer protocols must normally continue to cater for the massive existing infrastructure of nodes that operate on older ones and so the solution is often to extend the functionality of existing protocols.

It is also necessary to decide at what point encryption will be applied in a network, bearing in mind that there will be many points of vulnerability, from wiring cabinets to wireless signals to the packet switching network itself. With **link encryption**, every vulnerable link can perform encryption and decryption. This causes an added overhead for packet switching as packets must be decrypted and encrypted again before they can be forwarded and each pair of nodes must share a key for this to be possible. (Sharing keys between all the nodes is less secure.)

Alternatively, one can use **end-to-end encryption** where only the end-nodes – sender and ultimate receiver – must be able to perform encryption and decryption. Any nodes along the way transmit data unaltered. In this case, the header cannot be encrypted, and this means that analysis of traffic remains a threat.

A more secure approach involves using both link and end-to-end encryption, using a single end-to-end encryption key for the data and link encryption keys for headers.

These services are provided by **secure channels**. A secure channel is a communication channel between a pair of processes that can reliably identify each other, that is, they must be able to authenticate each other. This means that each party can check

Note that export restrictions may apply to some algorithms!

whether any requests should be allowed. In addition, secure channels provide confidentiality and integrity services, and a time-stamp is used to prevent replay or alteration of the order of messages.

In our context, the most significant protocols are HTTP and TCP/IP. In the next subsection, we will briefly discuss modified versions of these protocols (SSL and IPv6) that help to make this kind of communication secure. Both SSL and IPv6 are implementations of the idea of a secure channel.

4.2 IPv6

Unit 5 introduced you to the OSI (Open Standards Interconnection) model and its relationship to the operating system and some common protocols. Figure 7 is a reminder of a Unit 5 figure of the OSI model and its relationship to the operating system and some common protocols.

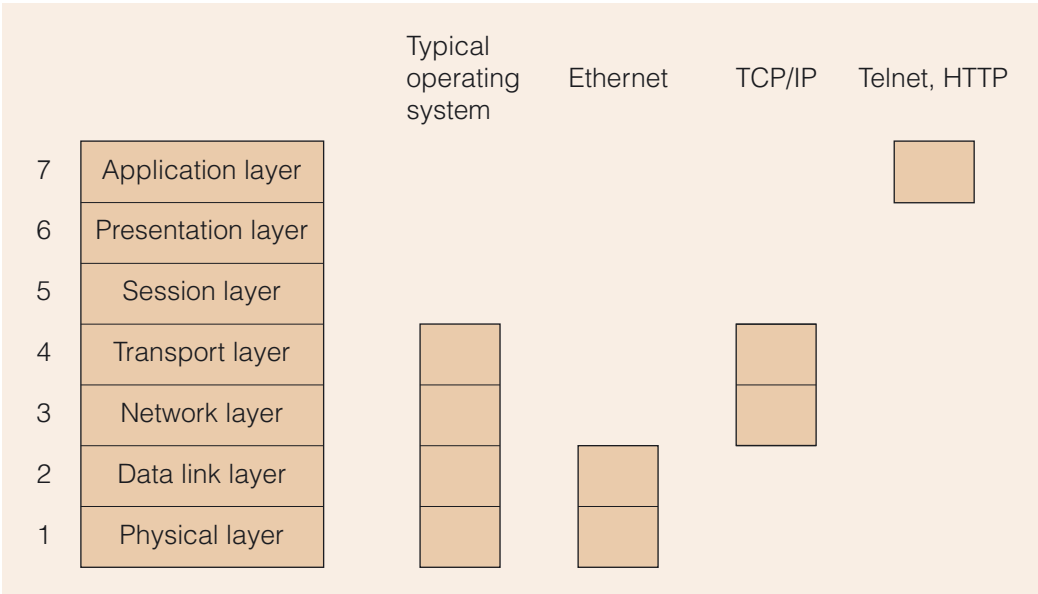


Figure 7 Mapping an operating system and protocols onto the OSI model

The basic unit of communication is the packet, which is a block of data composed of a header and a body (data), followed by an end marker. Packets travel across the data link layer. The packet header includes information enabling it to be sent to its intended destination, specified using an IP address, part of the IP. The IP, which operates in the network layer, ensures that the packets are sent to their correct destinations.

TCP operates on top of the IP, in the transport layer, and provides a checksum as an integrity check, but there is no guard against intentional alteration of packets and no way to verify an IP address. This means that fabrication of packets is a particular problem, which is the basis of some kinds of denial of service attack leading to interruption of network services.

At the network communication level, the normal solution to creating a secure protocol is to build on TCP/IP, which forms the basis of most internet communication. This means that only the end points of the communication need to be able to interpret the higher-level protocols. This is the purpose of IPv6, the newest version of the IP.

IPv5 doesn't exist.

IPv6 became a draft standard in 1998, but due to the investment in existing infrastructure based on IPv4, it is still only gaining ground slowly. IPv6 provides for many more addresses than IPv4 (at the time of writing still the most commonly used protocol), which it includes as a sub-range. In addition to requiring the use of security features, it

provides better support for the present wave of computing, including mobile and ubiquitous applications and networked entertainment. For example, packet headers may contain information about the quality of service required in order to judge how the media data should be compressed.

The security mechanisms used in IPv6 are described within the IPSec (IP Security) architecture, which IPv6 implements, and are extensions to IP header capabilities. These security mechanisms are described below.

1 Authentication header

The authentication header provides support for integrity, using a checksum of the contents of a packet. A hashing algorithm is used, whose results depend on a secret key known only to the sender and receiver. This can also be used to authenticate the sender.

The header also provides a counter that is incremented for each new packet (up to 2^{32} , at which point a new session must be negotiated). This number is used by the receiver to counteract replay attacks.

2 Encapsulating security payload

This protocol can be used to provide confidentiality by encrypting a packet and includes a checksum for verifying integrity. Optionally, authentication may also be applied in this header

These features allow distributed applications such as remote login, client-server communications and email to be secured at the packet level. Because these features are implemented below the TCP level, they are transparent to applications that depend on TCP for communication.

MD5 is a hashing algorithm applied in this context.

Similar features may also be implemented in a firewall or router.

4.3 Secure Sockets Layer

In the context of web communications and ecommerce, concerns for the security of credit card numbers and other data drove the development of the Secure Sockets Layer (SSL) protocol. You can tell this protocol is in use when you see a URL beginning with the protocol `https`, which indicates that the HTTP protocol is being layered on a secure sockets implementation. (However, this does not say anything about how the data is stored or used on the web server.)

SSL was originally developed by the makers of the Netscape web browser for transmitting private documents via the internet. It is now widely deployed for ecommerce and is the *de facto* standard for secure communications on the Web. The protocol makes use of TCP to provide both authentication and privacy through encryption. The protocol employs a **handshake**, that is, there are negotiations between the communicating parties to establish parameters for communication. At various points of the handshake, a step may fail and the connection can be dropped.

Initially, when a client requests a secure web page, it sends a message indicating what forms of communication the client's browser can support. Client and server must be able to communicate using an encryption algorithm that both support.

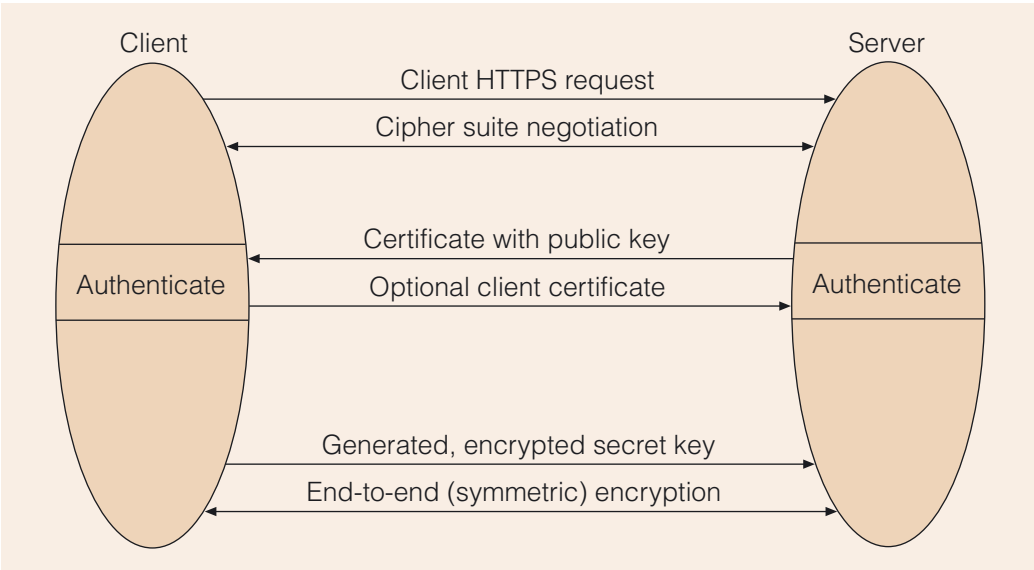


Figure 8 A typical SSL transaction

Authentication of both client and server can then take place through an exchange of certificates. Normally only the server authenticates itself to the client, but the server can also request a client certificate if it needs to authenticate the client, which may want to access a resource for which authorisation is required.

Once client and server have validated each other, a shared means of encryption is established, and a public key-based protocol can be used to establish a secret session key. The secret key is then used to encrypt communications, according to the chosen encryption scheme (Figure 8).

Diffie–Hellman is an example of such a key-exchange protocol.

A number of other techniques are applied, including compression and, at the end, a hash of all exchanged data to check the integrity of the communication. A nonce value can also be employed to prevent replays. In addition, the cipher in use can be renegotiated from time to time, if this is desired. For example, a message authentication code could be requested so as to ensure integrity of communication at some point.

SSL can also be used underneath other application protocols to create protocols such as FTPs for secure FTP, SMTPs for secure mail transport and LDAPs for secure lookup.

As there are several versions of SSL, the Internet Engineering Task Force is now in charge of the development of the Transport Layer Security (TLS) protocol, a standardised version of SSL.

SSL and TLS are both supported in core Java and in the Java EE platform through the Java Secure Sockets Extension (JSSE) packages, and can be called on to ensure confidentiality and integrity when data travels across boundaries between components over networks. To use SSL or TLS, a server must have a certificate, for example, a certificate associated with its IP address.

4.4 Firewalls

In a building, a firewall is designed to prevent a fire from spreading. When the fire reaches the wall, it cannot go any further, because the wall has been designed to resist the heat. This allows damage to be contained. This idea of containment is reflected in the name of the communications filter in computing called the **firewall**, in this context meaning a way of identifying and allowing or rejecting network traffic to or from a system.

Firewall software often shows the IP addresses of nodes that have been scanning your computer for open ports.



Firewall testing.

Once a computer is connected to a network, it is potentially open to communication from many other systems, some of which may be looking for vulnerabilities. The job of a firewall is to secure the computer's ports and to reject communication from unknown or untrusted sources, thus regulating the nature of network traffic entering and leaving ports on your computer. On a larger scale, a firewall protects a local network from the internet at large, providing a single point at which security and auditing can be imposed.

Hardware-based firewalls use a relatively secure operating system, carefully configured to avoid intrusion and alteration, and may support secure protocols to provide security for all traffic leaving and entering an organisation's local network. This is illustrated in Figure 9, which shows that some traffic arriving from the internet is not allowed to pass through the firewall.

Naturally, this does not protect against traffic that bypasses the firewall (for example, a dial-up connection may bypass the firewall) or from an internal attacker.

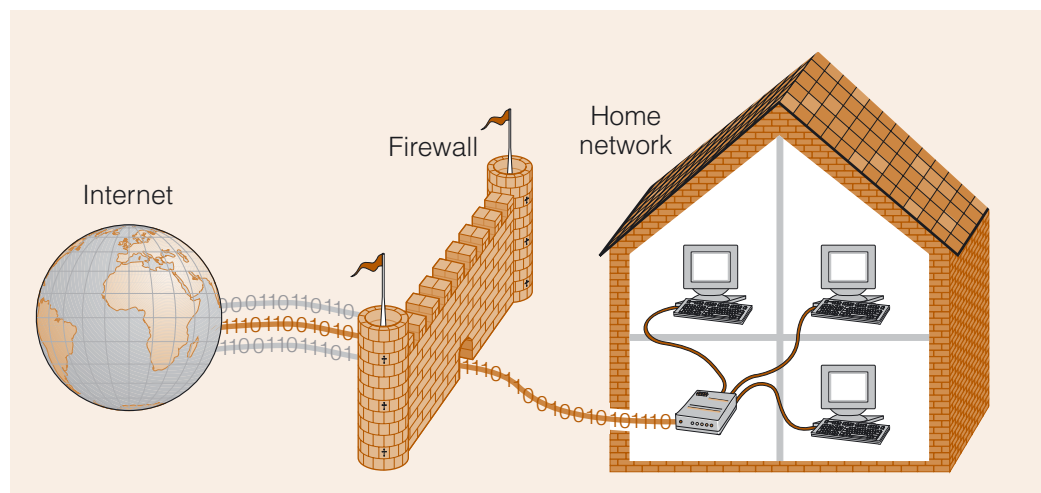


Figure 9 A firewall controls traffic between the internet and a local network

Packet filtering is the most basic firewall technique. A packet filter examines incoming or outgoing network packets against a set of rules, for example:

- ▶ whether they originate from a trusted network address;
- ▶ what protocol each is using, for example FTP may be blocked;
- ▶ whether they contain particular banned words;
- ▶ whether they are attempting to connect to a particular port.

It then accepts or rejects each accordingly. Typically the firewall will allow only traffic that the firewall administrator has explicitly allowed. For example, on a web server, port 80 traffic would be allowed but other traffic might not be. While this is a simple approach, it requires the manual creation of many rules.

Stateful packet inspection is an advanced firewall technique in which the state of each communication between the two sides of the firewall is tracked and used to decide which packets are legitimate. For example, if an outgoing request requires a response of a particular type, then that packet type will be allowed back in.

A firewall type known as an **application gateway** acts as a proxy server and provides a layer of authentication between the user and the desired service. In this way, unauthorised use of the service can be rejected. Such a firewall can also check incoming packets for unauthorised or dangerous content.

A benefit of gateway firewalls is in implementing virtual private networking. A **virtual private network (VPN)** allows an organisation's local area network to authenticate users who are routing communications over an outside network. The firewall will typically use IPSec to authenticate the user, who can then enjoy full access to the local network from a distance, with encrypted communications. In other words, a VPN allows private communication over a public network. The connection may be established initially by dial-up, broadband or another form of network connection, such as a leased line.

VPN is a form of **tunnelling**, that is, it uses a protocol that is wrapped in another protocol. This allows communication over a network that does not support the wrapped protocol directly. End-to-end encryption takes place between the two ends of the tunnel, so creating a private network of sorts. Unlike with a physical connection, it is easy to add users to a virtual connection, provided that they have a connection to a public network, so VPNs scale much better than leased lines.

When firewalls restrict too much traffic, they can be seen as a nuisance by users. A common case is that the firewall may block traffic on a port used by a peer-to-peer file-sharing program. To get around this, users may tunnel through the firewall. Firewalls will typically allow traffic through port 80 for web communication, and this means that any traffic wrapped inside an HTTP packet is allowed through unless the firewall inspects the packet contents. The receiving end can be primed to unwrap the packet and retrieve its contents. Potentially, such traffic could be dangerous. This is a case of a security measure interfering with usability (even if it is doing so for good reasons), but users may deliberately find ways around such measures. This means that firewalls, like other individual measures, cannot be seen as a panacea for security problems.

SAQ 6

What is a secure channel?

ANSWER.....

A secure channel is a communication channel between a pair of processes that can authenticate each other and provides confidentiality and integrity services, including time stamping.

SAQ 7

Give an example of a handshake in SSL.

ANSWER.....

Cipher negotiation and certificate exchange are examples.

Exercise 13

Why would SSL use a public key-based protocol to establish a private key for communication?

Discussion

Key distribution is easier with public key cryptography. The public keys can be verified by using a certifying authority and digital certificates. The private key obtained in this way results in faster communication for the bulk of the transaction.

5

An overview of Java platform security features



Java security overview.

The first half of this unit has concerned itself with security in general. For the remainder of the unit we are concentrating on the Java platform and showing how it has been designed with security in mind.

In this section, we discuss:

- ▶ features that are part of the design of the Java language, including Java's handling of types and memory;
- ▶ features within the JVM, for example, the mechanisms involved in checking code incorporated into the virtual machine;
- ▶ access control mechanisms within core Java (and hence also enterprise systems), that is, mechanisms that decide what code is authorised to do.

You will already be familiar with many of the ideas presented here, but considering them in combination helps to show that Java is a mature and relatively secure platform.

5.1 Java language features

In this subsection, we look at how the Java language itself is engineered with security in mind. Java provides good type safety, object orientation and other features that help programmers to express their intent clearly. This expressiveness, coupled with careful type and runtime checking allows many errors to be caught and makes it harder to subvert the intentions of the programmer.

The Java compiler also enforces catching or the explicit declaration of checked exceptions, which may help to avoid unhandled errors that might otherwise lead to unpredictable program behaviour.

Typing

Type safety refers to the ability of a language to detect operations that are not correct for a particular type. We take the view that typing is a way for a programmer to express their intent regarding the use of variables and values, and that therefore detecting unusual uses of types is a desirable thing. Type safety also ensures that at any given time the value of a variable will be within the range of values that is legal for the type. From this point of view, the variety of available types contributes to the expressiveness of the language and therefore the extent to which a programmer is able to express their intent. We start by observing that Java is both **strongly typed** and **statically typed**.

By strong typing, we mean that every variable, expression or literal value (such as a character or integer) has a type and that it is not possible to assign expressions or values of one type to another in an arbitrary way.

In a dynamically typed language, types are determined at runtime.

A statically typed language is one in which the types of variables are determined at compile time rather than at runtime. Because the type of a variable is fixed on creation, typing errors can be caught *early on* in the development process. For example, in Java it is not possible to convert `boolean` values to or from any other type, and reference variable types can be converted only to related types (where an inheritance or implements relationship exists). Likewise, a method can be invoked only on a suitable reference type.

Although in some cases types can safely be converted automatically, in others this would result in a loss of precision or in unexpected changes to values. To allow these potential errors to go uncorrected, as in weakly typed languages such as C or JavaScript, could lead to runtime errors or to incorrect values being propagated through a system undetected. Java's typing guards against these issues.

Memory access

Perhaps the most commonly observed software security issue is the failure to check the size of information being written to a buffer. This means that memory contiguous to the buffer can be overwritten, known as a **buffer overflow** error. This can result in a program in memory being overwritten by other code. When this happens accidentally, it is likely that memory will be corrupted and cause a program crash. More significantly, an attacker aware of such a vulnerability may be able to cause program execution to jump to a chosen address, simply by causing a large argument to be passed to a method.

In Java, array indexes are checked at runtime, meaning that it is not possible to access memory outside the array arbitrarily. Thus, Java's handling of memory is relatively secure.

Memory management

In some languages, like C/C++, a programmer must manage memory using methods operating on **pointers**, which are numerical references to addresses in memory that can be manipulated arithmetically. This means that a pointer can be incremented, decremented, added to, subtracted from, or even reassigned an arbitrary value. As a result, a pointer can be made to refer to an area of memory that it should not do, and this might result in the execution of incorrect or malicious code.

Pointers can also lead to hard to detect errors such as:

- ▶ using memory that has already been freed, which may lead to corruption when the memory is subsequently allocated;
- ▶ allocating the same memory to more than one purpose, leading to corruption of memory;
- ▶ attempting to deallocate memory that is still in use;
- ▶ forgetting to deallocate memory, or deallocating too little memory, known colloquially as a **memory leak** – this eventually leads to an out-of-memory error and is a particular issue in embedded systems, where software may be left running continuously;
- ▶ attempting to use a null memory reference, leading to exceptions.

The Java language employs **references** instead of pointers. Although these terms are sometimes used interchangeably, the distinction is an important one. Java references do not allow arithmetical manipulation of memory addresses. References are fixed by assignment and cannot be changed in arbitrary ways. It is also not possible to write to an area of memory directly. Memory allocation is automatic, and deallocation is provided by the garbage collector, which reclaims unreferenced blocks of heap memory.

Modifiers

Through appropriate use of access modifiers such as `private` and `protected`, a programmer can express intention regarding appropriate access to instance data and methods within a class.

Typically, data is implemented as `private` and is made accessible through a method, where needed. Both compiler and runtime checks can take place to ensure that these intentions are not violated. Of course, Java programmers still need to be alert to potential pitfalls. For example, returning a reference from a method leads to a client having access to the referenced object's public methods, and this may not be desirable. Instead, a clone (copy) object could be returned, or data could be returned in another form.

Constants

By applying a `final` modifier to a variable, method or class, it can be protected against changes at runtime or through inheritance (by a malicious or careless programmer). For example, setter methods can be made `final` to avoid a subclass subverting correct initialisation of an object. A class can be made `final` to prevent it being subclassed, as is the case for the `String` class. A parameter to a method can be made `final` to prevent it being changed.

Java strings are immutable – once they are created, they cannot be changed. In combination with `final` references, this can help prevent some runtime errors, which might be exploited to gain control of a machine. For example, a `final` string's length cannot be increased or arbitrary characters introduced to try to cause an application to crash. If a string reference is `final`, it cannot be made to point at any other string. Conversely, strings are not recommended for storing login and password information, as they cannot be 'erased' as readily as data held in arrays of primitives (such as characters).

`final` methods are heavily used within Java's security checking mechanism to access operating-system-level features.

5.2 The Java Virtual Machine

Most web browsers now use a Sun Microsystems-provided JVM plug-in rather than a browser-specific virtual machine. This means that most concerns about vulnerabilities in individual virtual machines have been solved. Vulnerabilities at the virtual machine level are generally hard to exploit, and can be fixed by patching in the same way that operating system issues are addressed.

In this subsection, we discuss load-time security features.

The class loader – good fences make good neighbours

As you saw in *Unit 4*, all classes must first undergo **loading** into the JVM, using a class loader, which loads classes from authorised locations.

Before passing a class to the runtime environment, a class loader:

- ▶ loads classes from an input stream, usually a network location, translating it from raw bytes into an internal (virtual machine-specific) representation of the class;
- ▶ verifies that bytecode conforms to expectations;
- ▶ defines namespaces (the set of packages that are in scope) for classes;
- ▶ defines what permissions a class will have, depending on where it came from, and whether it is signed, which can be used by the runtime environment to determine what the code is allowed to do.

There is a class loader known as the **primordial loader** whose job it is to perform the first step, called bootstrapping, which involves loading the first class as well as the Java API classes. New classes are loaded thereafter when a new object is created, or there is

a static reference to an unloaded class. Although user-defined class loaders (which can perform specialised tasks, such as loading applets or RMI classes, or reading encrypted class files) may also be used, only classes loaded by the primordial loader are considered trusted, and the primordial loader cannot be overridden.

Each class loader looks after a separate namespace. Standard API classes are placed in a trusted namespace. The virtual machine maintains the boundaries between these namespaces, so that classes with the same simple name in different packages cannot be confused with each other. This means, for example, that a user class `mypackage.String` is not substitutable for `java.lang.String`; therefore, user classes cannot undermine the behaviour of standard classes or other user classes.

Web browsers use a different instance of the class loader object to download every applet. This maintains separate namespaces for different applets, again maintaining boundaries and avoiding conflict between code from different sources. Failure to do this might result in one running applet intercepting messages to another, by virtue of having a class with the same name and same method signature.

The bytecode verifier

It cannot be assumed that a class file was created by a correctly operating Java compiler, or has not been subsequently corrupted or deliberately altered. Therefore, before a class can be incorporated into the runtime state of the virtual machine, which we call **linking**, its bytecode must be verified.

The bytecode verifier is invoked as part of the class loading procedure and checks that bytecode from outside the core API conforms to various rules. If it does not, the virtual machine throws a `VerifyError`, which will cause the system to terminate if it is not caught.

The details of the operation of the bytecode verifier are complex and beyond the scope of this course. However, as was mentioned in *Unit 4*, a number of important security checks are made. Verification of code happens in several passes. First an integrity check is performed, to ensure that a class appears to be correctly formed. The sizes of structures inside the file are checked to guard against errors or alterations of the class file.

Other semantic checks include:

- ▶ only legal bytecode operations may appear;
- ▶ there must not be any attempt at pointer arithmetic;
- ▶ access restrictions such as `private` or `protected` must be honoured;
- ▶ `final` methods must not be overridden;
- ▶ `final` classes must not be subclassed;
- ▶ class and variable types must not be violated;
- ▶ methods must be invoked with appropriately sized and typed arguments;
- ▶ operators must be used with appropriate types;
- ▶ a data flow analyser checks the code against a number of constraints to protect the virtual machine stack from overflow or underflow.

Bytecode obfuscation

It is worth mentioning in passing that bytecode can be decompiled, allowing an attacker to reverse engineer code and look for weaknesses, hence class files are a potential security hole.

There are many ‘bytecode obfuscation’ tools available to take readable code and change its outward appearance while maintaining its operation from a ‘black box’ point of view.

Code may be run using an application server if secrecy of the code itself is important.

SAQ 8

Why would a class loader check for overriding of `final` methods, when this check is already performed by a compiler before producing a class file?

ANSWER.....

The class loader is examining bytecode, which could have been altered since compilation.

5.3 Authorisation

In Section 6, we will show how access control can be linked to user identities.

Our next concern is for what code is authorised to do once it has been incorporated into the virtual machine. In particular, this section discusses how the security manager controls access to resources and checks the legality of code where necessary, in terms of a security policy.

Authorisation mechanisms in Java have been developing for some years now. Initially the distinction was made between local code and code retrieved from a network. Local code was trusted, while remotely loaded code was considered unsafe, because its origin was uncertain. The unsafe code was said to run in a ‘sandbox’, which is meant to suggest a play area (as in a child’s sandbox) where no harm can be done. Code in a sandbox cannot perform risky operations such as writing to the local disk.

Subsequently, this model was seen as too limiting. Since Java 2, programmers have the ability to provide fine-grained control over what code from any location is allowed to do. However, security checks are only performed if a security manager is running. By default, the security manager is running for applets, servlets and other components in containers, such as EJBs. More generally, as you saw in *Unit 7* with RMI, in order to run applications with a security manager, either the application itself must programmatically set one (via the `setSecurityManager` method in the `java.lang.System` class), or the virtual machine must be invoked with an appropriate argument.

In fact, the `SecurityManager` class approximately follows a design pattern called the **singleton**, in which only one object of the class implementing the singleton pattern can exist at a time. Once the security manager is in effect, the `SecurityManager` constructor will check to see if further instantiations are allowed by a security policy. The default is that only one may exist at a time.

SAQ 9

Why would a security policy require that the security manager could only be set once?

ANSWER.....

This means that another manager cannot be substituted, so the ways in which checks are performed cannot be altered.

Protection domains

We now discuss the refinement of the sandbox model in the interaction of security features known as a **protection domain** that controls authorisation of what code can or cannot do. This is shown schematically in Figure 10.

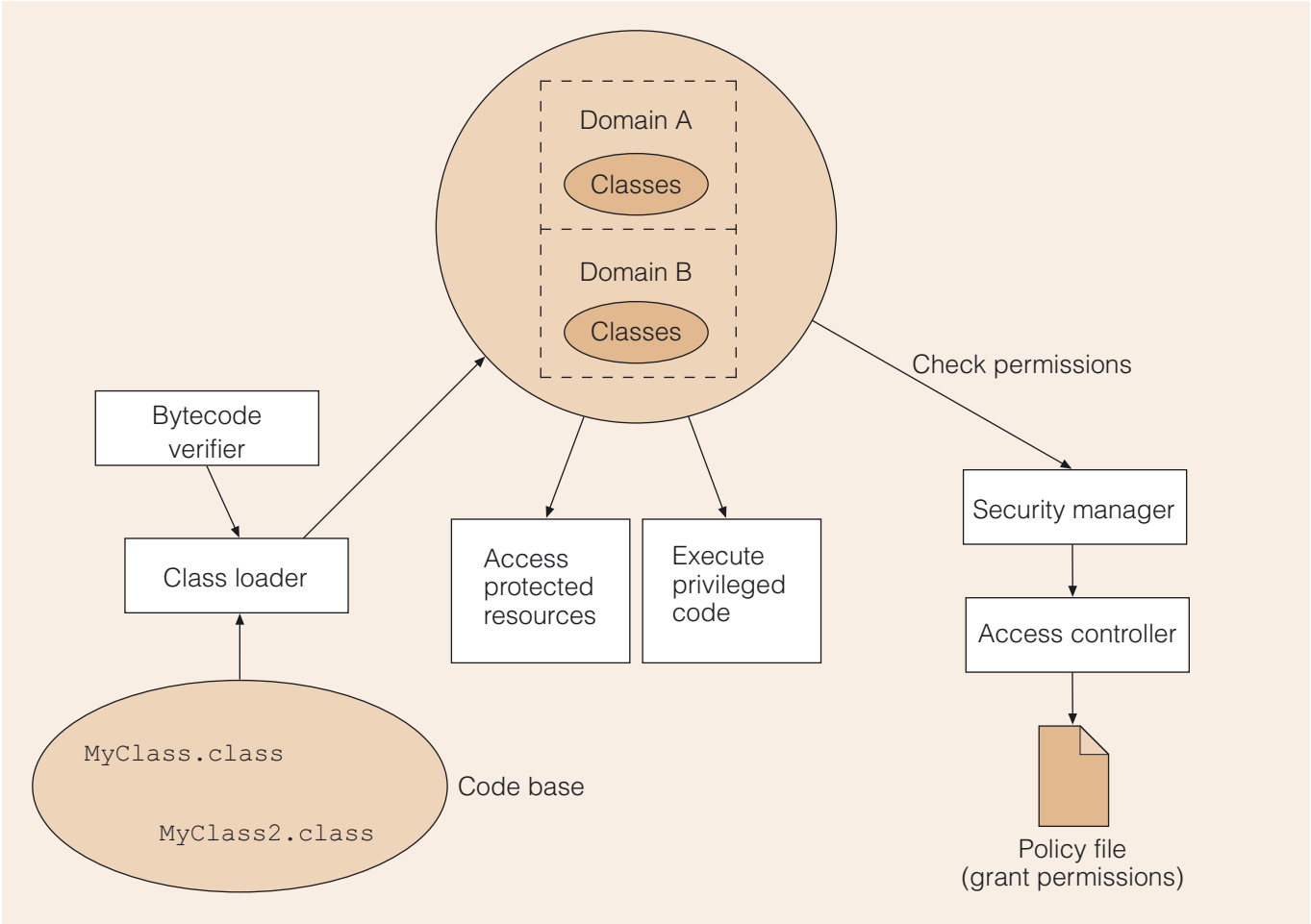


Figure 10 Protection domains

Classes originate from one or more **code bases**, which indicate the location of compiled code.

Classes then pass through a class loader, which maintains boundaries between trusted and untrusted code, and verifies that classes conform to various rules, using the bytecode verifier.

Once linked into the virtual machine, each class belongs to a **domain**, which is a group of classes with the same **permissions**, such as read or write access to files.

Code permissions may be assigned according to:

- ▶ where the code originated (its code base);
- ▶ an identity associated with the code (discussed in Subsection 6.1), for example a user identity, known as a **principal**;
- ▶ a digital signature associated with the code (a certificate).

Not all of this information may be available and all or none of this information may be used in assigning permissions to a class.

Trusted classes are part of the **system domain** and are allowed to access system resources, such as file I/O and network communications, and to execute related (privileged) code. Other classes lie in **application domains**.

Sets of permissions constitute a **security policy**. There are default security policies for application and system domains, but these can be configured. As we will see below, in 'Describing permissions', the policy is usually constructed from one or more policy files. There can be only one policy in effect at a time and so this is sometimes referred to as the policy decision point.

A **security manager** (an instance of the `SecurityManager` class) performs checks on the validity of actions requested by code. This usually happens via an **access controller** (using `AccessController` class methods) and is also known as the policy enforcement point.

The protection domain system underpins security in both core Java and Java EE. For example, components such as applets can be granted permissions using the same approach.

The security manager

Some examples of checks performed by the security manager are authorisation to:

- ▶ access the event-handling queue (which handles GUI events);
- ▶ access sockets (host and port numbers) for reading or writing;
- ▶ access files – for example, read, delete or execute them;
- ▶ access system properties (such as the version of Java running);
- ▶ create class loaders;
- ▶ create operating system processes (for example, launch a help window from an application, or open a new browser);
- ▶ alter threads' states within the virtual machine;
- ▶ access particular libraries;
- ▶ access the system clipboard (where copied material is held);
- ▶ perform security-critical operations, such as accessing a list of permissions a class has, or running code with certain privileges enabled.

As you can see, the coverage of the checks is very broad, and we have shown only a sample. When such operations are requested by application code, the security manager consults the security policy in effect, to check whether the operation is allowed or not. For this purpose, there are a number of methods defined in the `SecurityManager` API with names beginning with `check`; for example, `checkRead` to see if a file may be read. The security manager then either authorises the action silently (i.e. without any outwardly visible effect) or denies it by throwing a `SecurityException` (or subclass).

Since Java 1.2, the security manager usually makes use of the `AccessController` class to perform its checks. This class is not instantiated, but provides a number of static

methods intended to provide a good default security-checking algorithm. The `SecurityManager` has been retained mainly for backward compatibility.

Let us briefly examine how this works in practice, using a request to read a file as an example. When a particular class attempts to read the file, at some point it will cause a `FileInputStream` to be constructed. The `FileInputStream` constructor code includes the following code segment.

```
SecurityManager security = System.getSecurityManager();
if (security != null)
{
    security.checkRead(filename);
}
```

Thus, the runtime system

- ▶ checks to see if a security manager exists;

and, if it does exist,

- ▶ checks if the calling code has permission to perform the requested action.

This in turn invokes the `checkPermission` method in the `AccessController` class to check whether the permission in question is allowed in the context in which this code is being executed, known as an `AccessControlContext`. If the permission does not exist, the access controller throws an exception. If it does exist, the method simply returns silently and the file is opened as requested.

Permissions pertaining to classes from each domain are carefully controlled. In particular:

- ▶ A class may not gain extra permissions as a result of being called by a class with more privileges.
- ▶ When a task involves executing code across a number of domains, normally the intersection of the permissions of all domains traversed by the thread produces the permissions in effect for the thread. More specifically, the `AccessControlContext` includes an array (representing a stack) of `ProtectionDomains`, and every `ProtectionDomain` must have permission to perform an action in order for it to be allowed.

Describing permissions

Permissions can be described:

- ▶ statically, using a default system-wide policy file and optional user policies;
- ▶ using a user-defined policy file as an argument to a JVM;
- ▶ dynamically, using predefined or user-defined permission classes so that an application can specify a `Policy` object with the desired permissions.

The location of the default system policy file is recorded in the master security properties file `java.security`. By default, there is also an optional (but empty) policy file in the user's home directory.

The default system policy is that all privileged actions are denied, apart from some frequently needed permissions that are deemed safe, such as the ability to read the local line separator and path separator properties and the version of Java runtime.

There are two exceptions to this:

- ▶ classes are allowed to read from their own directories (and subdirectories);
- ▶ classes in an application class path are allowed to terminate the virtual machine, so that an application can terminate itself.



Activity 10.5

The security manager.

In our activities, we will concentrate on the static description of permissions using a user-defined policy file passed to a virtual machine. A policy described in this way can be used to augment or to replace any default policies. (Dynamic permissions cannot be used unless the security policy grants permission to do so.)

Policy files

Each line in a policy file is called a **policy entry** and consists of a permission type, a **target** (for example, a folder or a port number) and in some cases an **action** (for example, read access) specified as a string. The possible actions depend on the target.

Policy entries may be applied to code from a particular code base, signed a certain way or associated with a certain caller. We discuss principals in Section 6.

For example, to grant permission to all code from the `http://www.M362MusicStore.com/` code base to read the `C:\MusicStore\music.dat` file, the following may appear in the policy file.

```
grant codeBase "http://www.M362Musicstore.com/"
{
    permission java.io.FilePermission "C:\MusicStore\music.dat", "read";
};
```

The `codeBase` is a URL, and so uses forward slashes, while the file path reflects the path separator on a particular operating system. The permission here is the `java.io.FilePermission` and the target is the `music.dat` file, while the action is `"read"`. (It would also be possible to specify a write permission as an action in this case.)

Our example shows granting permission to code from a particular code base, but the opening line of a `grant` clause may also specify that permissions are to be applied to a particular principal, for example, giving permission to the music shop owner to read the user's home directory, or it may specify that the permission applies only to appropriately signed code.

Defining your own permissions

Because you can define your own permission types, permissions can be attached to arbitrary actions. For example, you could create a class called `MusicDatabasePermission` as a way of checking if The Music Store's database may be accessed. You can then use a clause in a policy file to allow code from a certain code base, or code associated with certain principals or signed a certain way, to have this permission. You can then also write code to check if a caller has this permission in any code requesting access to the database.

The steps to use in general are as follows.

- 1 Define your own permission classes; for example, by extending the existing class called `BasicPermission`. This provides a target for the permission. Your permission class constructor can receive a string argument representing an action name. In this way, an arbitrary set of actions can be associated with the same target.
- 2 Optionally, define your own exception to describe the security problem that exists if a user attempts to perform the action without permission.
- 3 Add an entry to your policy file granting your permission where required; for example, to code from a certain code base. Any action can be specified using a string value.

- 4 In your application code, check that the caller has the permission to perform an action by constructing an object of your permission class and, using the `checkPermission` method of the `AccessController` class, throw an exception if the permission has not been granted.

The interaction of these features means that once code is linked into the JVM:

- ▶ communication with the operating system and protected resources is controlled;
- ▶ it is possible to describe what permissions should be applied to code in various contexts.

SAQ 10

State the two steps required to implement permissions-based security for an application.

ANSWER.....

- 1 A security manager must be installed for the application, either on the command line or in the code.
- 2 A policy must be specified, either dynamically (by executing some code) or statically (using default policy files or a specified policy file).

SAQ 11

Explain the purpose of the following entry in a policy file. Identify the target and the action.

```
grant codeBase "http://www.gggg.com"
{
    permission java.io.FilePermission "C:\database.dat", "read";
};
```

ANSWER.....

This is a policy entry granting `FilePermission` to code from the code base `http://www.gggg.com`, allowing that code to carry out "read" actions on the file `database.dat`.

Exercise 14

- (a) In the code sample below, part of a `main` method in an application called `MyApplication`, line 1 throws a security-related exception, while line 2 does not. Suggest a likely reason for this.

```
FilePermission fp = new FilePermission("C:\database.dat", "read");
AccessController.checkPermission(fp); //1
Scanner sc = new Scanner(new File("C:\database.dat")); //2
```

- (b) On examining the permissions assigned to the context, we see that the following are allowed.

```
java.lang.RuntimePermission exitVM
java.io.FilePermission "C:\MyApplication\build\classes\" "read"
```

Explain why these permissions are safe.

- Discussion
- (a) The first check here (line 1) is against the policy decision point. Line 2 will require permission to access the file (the same check as in line 1) when a security manager is running. Since line 1 throws an exception we can conclude that the context does not have permission to perform the "read" operation. Therefore, if a security manager is running, line 2 will also throw an exception. However, given that line 2 does not throw a security-related exception, you can infer that no security manager is running and policy enforcement is not being applied.
 - (b) The first permission simply allows the program to terminate; the second says that the application can read files in its own class folder (but not, for example, c:\database.dat). These permissions are allowed to all applications by default.

Exercise 15

How might an applet use the security manager to probe your system's security?

Discussion

An applet could probe your security by attempting a restricted operation and seeing if a `SecurityException` is thrown. If the applet catches this exception, the client may not be aware that anything untoward has taken place.

6

Java authentication and authorisation

As you have seen, cryptographic services (and hence confidentiality, non-repudiation and integrity) are well provided for in Java through the Java Secure Socket Extension (JSSE) and the Java Cryptography Extension (JCE) packages. In the remaining sections, we will be concentrating on other key security services and in particular on authentication and authorisation mechanisms.

In Subsection 5.3, we saw how it is possible to use a security policy to specify actions that classes are allowed to perform. This might be considered a way of protecting the system from untrusted code.

However, how is the system to be protected from its users (or users from each other)? We mentioned earlier that permissions could be associated with an identity attached to a class. In this section, we elaborate on how this is achieved, including how it can be used to provide user-based authorisation.

One way in which this can be achieved on the client side is using the Java Authentication and Authorization Service (JAAS). JAAS has been part of the core Java distribution since Java 1.4.

6.1 Principals and subjects

To explain how this protection works, we first need to define some terms associated with authentication.

A **subject** is an entity that may be authenticated; for example, it could represent a person, an organisation, a node, a piece of software, a business or a service. Authentication takes place by some accepted means; for example, using a public key, password or certificate.

A subject may therefore own some security-related information, known as its **credentials**, such as passwords, certificates or cryptographic keys, which may be publicly accessible or private to the subject.

Once authenticated, a subject is represented by at least one principal. The set of principals associated with a subject can be used to grant permissions; that is, to decide whether a subject's requested actions should be authorised or denied. In effect, a set of principals represents the identity of a caller, whether that caller is a user, a piece of software or something else.

The ideas of subject and principal are quite general.

- ▶ A subject could represent one individual with multiple principals (for example, multiple login names, credit card numbers or membership numbers). That is, a subject could represent a single user who accesses multiple systems using different login credentials, or a single user who accesses multiple systems via a single authentication.
- ▶ An authenticated student at a university might be represented by a subject with two principals – a personal username and the generic principal named "student". In this way, all students might be authenticated to the university library as the generic "student". This is also known as role-based access control.

- ▶ A subject could represent a service. The service provider may present a certificate to prove its identity, and the client can check the certificate and hence authenticate the service provider. A principal name such as "service.shop.com" could then be used to represent the service provider.

Principals and policies

Once a subject is authenticated, its principals and credentials can be made available to other system components for the purposes of authorisation.

Principal information can be used programmatically – you can directly access the subject's public credentials and principal information in order to make a decision in code.

A more general approach is to specify in your security policy file that certain permissions are granted to particular principals. As before, this could involve writing your own permission types, or you could use predefined permissions such as `FilePermission`. The policy information is then used by the security manager in the usual way to decide whether an action should be allowed or not.

A policy file can contain a statement similar to the following.

```
grant principal Principal_class "principal_name"
{
    permission java.io.FilePermission "C:\database.dat", "read";
};
```

Such a grant statement can also depend on the code base or how the code is signed.

SAQ 12

In the following scenario, identify possible subject(s) and principal(s).

Jane Doe wishes to make an online purchase. Jane will first need to authenticate herself to her computer by logging in. Once online, she logs in to her shopping website account and makes a purchase using her credit card. Later she visits the website of the department of motor vehicle licences and enters her driving licence number to check that her contact information is correct.

ANSWER.....

The user Jane is the subject. The principals are her user IDs for her computer and for the online shop. (The passwords in each case are credentials used to authenticate her.) Another principal is the driving licence number Jane used to login to the motor vehicle licensing website.

The various JAAS-related classes are located under the package `javax.security.auth`.

6.2 Login context

JAAS provides a `Subject` class and a `Principal` interface and allows for a variety of authentication mechanisms.

An object of the `Subject` class is capable of storing:

- ▶ a set of private credentials and a set of public credentials – because the nature of credentials varies so greatly, any class of object can be used to represent them;
- ▶ an array of one or more `Principals`, each identified by a string.

A new `Subject` will not automatically have initialised this instance data to contain anything useful. Remember that a subject does not acquire any principals, which represent its identity, until it has been authenticated.

You can think of an authenticated `Subject` as a collection of principals; that is, a collection of identities for the subject.

A principal is an instance of an implementation of the `Principal` interface, a simple interface having `equals`, `hashCode`, `toString` and `getName` methods. The `getName` method simply returns a `String`. (The other methods are inherited from `Object`, so do not strictly require implementations.)

JAAS provides a separation between the application code and the code performing the authentication. The whole system is highly configurable and 'stackable' in that different authentications can be layered on top of each other. The higher-level system need not change if the configuration or manner in which authentication is performed changes. New forms of authentication may be invented and plugged in transparently. The programmer need not know how to deal with the intricacies of individual authentication measures.

JAAS is, in fact, an implementation of the Pluggable Authentication Module (PAM), a specification for authentication produced by Sun Microsystems.

Figure 11 illustrates how different authentication mechanisms can be 'plugged in' to an application using JAAS.

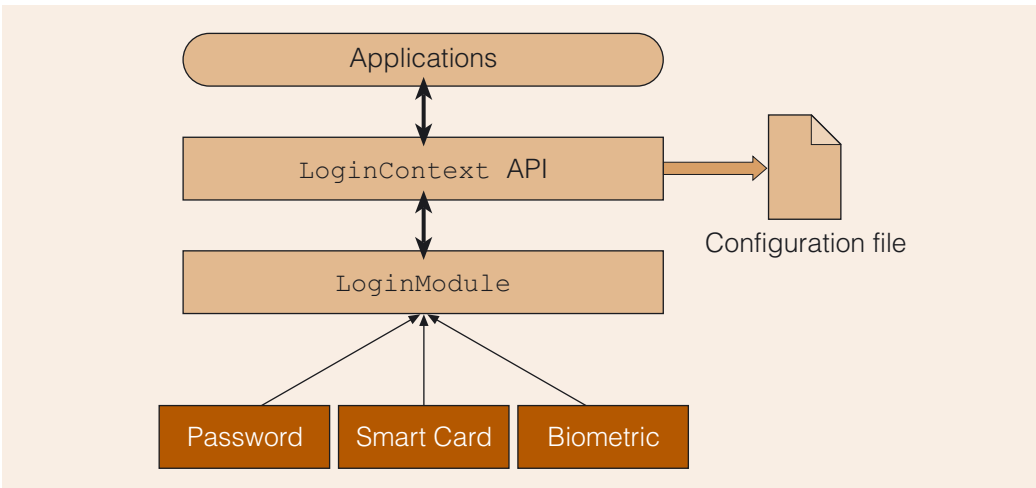


Figure 11 Pluggable Authentication Modules

Let us assume that we are authenticating a user. In order for application code to do this, it needs to create and authenticate a `Subject` object. Code can then be run with the permissions associated with the subject's principals.

- 1 In order to authenticate a `Subject`, it is first necessary to instantiate a `LoginContext` and to specify the name of a login module (a class implementing the `LoginModule` interface) that will perform the authentication. The login context has a `Subject`.

By default, the login module to be used is specified using a configuration file. This means that different login modules can be plugged in to the same application to provide different forms of authentication and also that different applications may request different forms of authentication.

A system may need to perform a variety of authentications at different times, in different orders, all or some of which may be necessary in order to authenticate a subject.

For this reason, JAAS provides for different levels of authentication, named *optional*, *sufficient*, *required* and *requisite*. We will not concern ourselves with all the details of how these levels of authentication operate; however, authentications that are *required* or *requisite* must succeed. If no *required* or *requisite* modules are

A `LoginModule` must have a zero-argument constructor so that it can be instantiated by other classes.

specified, then at least one *sufficient* or *optional* authentication must succeed. This means that a variety of authentication approaches can be specified with different levels of stringency applied to their completion.

The configuration file uses a flag to indicate whether each listed authentication is *optional*, *required*, *requisite* or *sufficient*.

In addition, key–value pairs may be read from the configuration file and used within the login module; for example, to set debugging options.

- 2 The `LoginModule` class must implement several methods, including a `login` method.

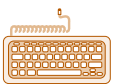
The `LoginModule`:

- ▶ requests credentials, for example, a username and password, data from a biometric scanner, data from a card, or more than one such operation;
- ▶ determines whether the login and password, thumbprint or other credentials are acceptable.

If successful, a `commit` method is invoked behind the scenes in the login module to store relevant principals and credentials in the `Subject`.

- 3 The authenticated `Subject` can be retrieved from the `LoginContext` by a method `getSubject()`. In this way, the subject's identity can be made part of the access control context and the subject's principals can be used to determine whether the subject has permission to perform any action it requests. (This can be used in addition to permissions based on where a class was loaded from, or how it is signed.)
- 4 When all work relating to the subject has been performed, a `logout` method can be invoked on the `LoginContext`, which results in the subject's principals and credentials being removed.

6.3 An example authentication



Activity 10.6 JAAS authentication.

We now present a minimal example of authentication, relating to an application we are calling the `MusicStoreClient`. We use a string containing this text as an index into a JAAS configuration file, to look up the form(s) of authentication required in the login context. The virtual machine needs to know where to look to find the configuration file (or it is passed as a parameter).

Here is an example configuration:

```
MusicStoreClient
{
    musicStore.StoreLoginModule required debug=false;
};
```

In the configuration above, `musicStore` represents a package. `StoreLoginModule` is a class implementing `LoginModule`, so it has `login`, `commit` and `logout` methods (amongst others). The module also states that this authentication is *required*. The text `debug=false` is an optional key–value pair, which might, for example, turn off debugging statements in the `StoreLoginModule`.

There may be other entries in the same configuration file, for example, there might be an entry for an `AdministratorClient`, and there may be an `AdminLoginModule` to provide a different form of authentication for the administrator.

The configuration file can also specify a default authentication to be used if there is no module matching the name used.

The client of The Music Store requiring the authentication constructs a `LoginContext` using the string identifier of the login module as an argument:

```
LoginContext lc = new LoginContext("MusicStoreClient");
try
{
    lc.login();
}
catch (LoginException)
{
    // Authentication failed.
}
```

During initialisation, the `LoginContext` finds the configuration entry `MusicStoreClient` in the JAAS configuration file to determine which `LoginModules` to load, each of which performs some authentication. (In our example, there is only one login module.)

Notice that if the form of required authentication changes, the client need not know – it will simply load a specified login module and invoke its `login` method.

Access control context

The access controller relies on a context to decide whether an action should be allowed or not. Previously we have assumed that this would simply be the context belonging to the thread that is executing and the permissions applied would be those associated with that context.

In this case we want to tell the security manager that we are doing work on behalf of a subject, and to use the permissions associated with that subject. Authenticating a `Subject` in a `LoginContext` does not achieve this, but the `Subject` class provides methods that can be used to perform work with the permissions associated with a particular subject.

Any work to be performed with special privileges is specified as a `PrivilegedAction`, which is an interface requiring implementation of a `run` method; the code in the `run` method is whatever code you want to execute with the authenticated subject's permissions. For example:

```
// We can now get the authenticated Subject from the context.
Subject sub = lc.getSubject();
// And we can execute code as this Subject.
Subject.doAsPrivileged(sub, new MyPrivilegedAction(), null);
//When we are finished, we can logout
lc.logout();
```

The access controller will then use the subject's permissions to decide whether any code in the `run` method should be allowed. The final argument to `doAsPrivileged` is the access control context with which the subject's permissions are to be combined. By making this `null`, the subject's permissions are combined with an empty context. In other words, only the subject's permissions are involved in deciding whether the action should be authorised. After the `run` method completes, the original context is in place once again. This is a feature to be used in a limited and careful way, and permission is needed to use security-critical methods such as `doAsPrivileged`.

There is also a method `doAs` which you can use to add the `Subject`'s permissions (or more precisely, permissions belonging to its principals) to the current context while executing a privileged action, so that both sets of permissions are taken into account.



Activity 10.7 JAAS authorisation.

Instead of `null`, you can also pass a previously stored context.

The approach we have just described depends on policy files. We may also retrieve the subject's principals and credentials and use them to decide whether or not to execute code, depending on whether a particular principal or credential is held by the subject.

SAQ 13

Why would you normally want to implement the `equals` method of an implementation of `Principal`?

ANSWER.....

You need to implement `equals` so that the system can compare one principal to another. If you do not do this, your principal will inherit `equals` from `Object`, and equality will be determined based on principal references rather than their contents.

SAQ 14

Why do you think the `PrivilegedAction` interface is required?

ANSWER.....

An instance of a class implementing the `PrivilegedAction` interface encapsulates and demarcates the work to be done with a set of privileges. This means that there is less chance of accidentally invoking code with certain privileges, and also it is clear when that set of privileges stops applying and we go back to using the thread's context.

SAQ 15

Distinguish between a subject and a principal.

ANSWER.....

A subject is an entity that can be authenticated, that is, a source of a request to perform some action. A principal is an identity associated with an authenticated subject.

SAQ 16

What is the purpose of a login module in JAAS?

ANSWER.....

A login module implements a method of authentication, such as biometric scanning or prompting a user for a login and password. A `commit` method is used to associate credentials with a subject.

SAQ 17

When would you use the method `doAsPrivileged`?

ANSWER.....

You would use this method when you want to execute code with the permissions associated with a particular principal. You can grant a principal permissions in a policy file, and this principal can be associated with a subject on authentication.

Exercise 16

When would you consider it safe to use `doAsPrivileged` with a `null` context?

Discussion

You can use this when you have some work to be done with a subject's permissions and can override the current context. In effect, this means that it must not matter how the point of code execution was reached. That is, it should not matter who, or what, called the code to invoke `doAsPrivileged`, it should only matter what the identity of the authenticated `Subject` is. An example is that an authorised application needing to access system fonts to render a document should be capable of gaining the permission to do so.

7

Containers and authorisation

In this section, we will concentrate on security needs of components in the middle tier of an enterprise system, that is, the web and business tiers. We begin with an overview of container security in the middle tier and then discuss mechanisms by which the middle tier can perform authentication and authorisation. Although JAAS has been integrated with enterprise systems, the details of how this is achieved have so far varied from one implementation of Java EE to another and so we will address only container-provided services.

7.1 Overview of container security

Java applets, servlets and EJBs are run within containers. Before such components are deployed, they are verified to be well formed and to conform to the Java EE specification. Enterprise-level code therefore receives security support from:

- ▶ the programming language and its design for security;
- ▶ the APIs, for example, the Java security architecture gives the freedom to choose cryptographic services from various providers – by-products of cryptographic services are confidentiality, non-repudiation and control over integrity of data;
- ▶ the virtual machine class loader and verifier;
- ▶ the `SecurityManager`, which runs by default within a container (referred to as system-level security in the context of containers) and accesses a security policy.

The container settings will determine the support provided by the server for security and other services. For example, containers can provide some auditing of traffic and transactions, so providing non-repudiation services. Because security behaviour is configurable by the container, the same component could have different privileges in different containers. Note also that having a container-managed security provides scalability and means that changes to security and other services can be easily adopted by application code by configuration of the container.

Once data travels across container boundaries, the container can provide support for transport-layer security (such as SSL). Message-layer security is also provided for, such as encrypted SOAP messaging, for web services. (SOAP and web services are discussed in *Unit 11*.)

Container authentication

Because there is a cost involved with authentication, in some cases it is performed only when required, when a protected resource is requested by a client (this is known as lazy authentication). For example, you may browse a website without authentication up to the point where you want to make a purchase.

In the following figures we will use the common scenario of a user authenticating to a website. However, you should bear in mind that a client is not necessarily a human user, and authentication may take other forms, such as the presentation of a certificate.

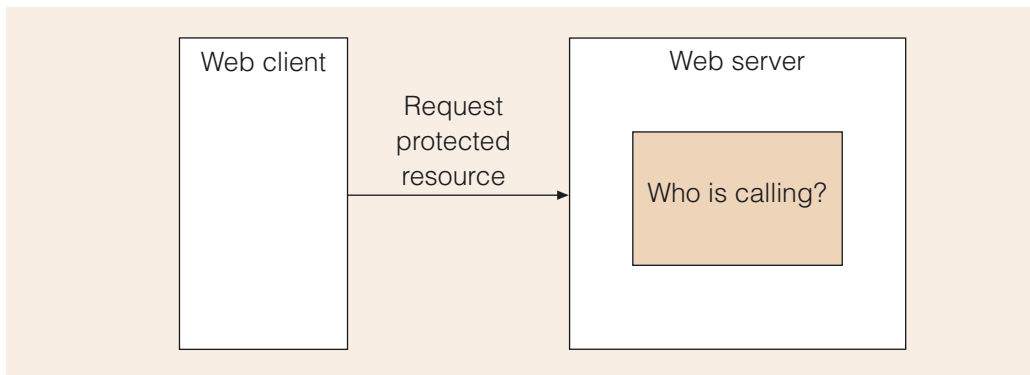


Figure 12 Requesting a protected resource

When a client requests a protected resource from a web server, the server detects this and, if the client has not been authenticated, the server invokes an appropriate authentication mechanism. This use case is illustrated in Figure 12.

At this point the web server challenges the caller to authenticate itself. The client then forwards its credentials to the server. If the subject has been authenticated, credentials can be associated with the caller on the server side as part of a session context, as shown in Figure 13.

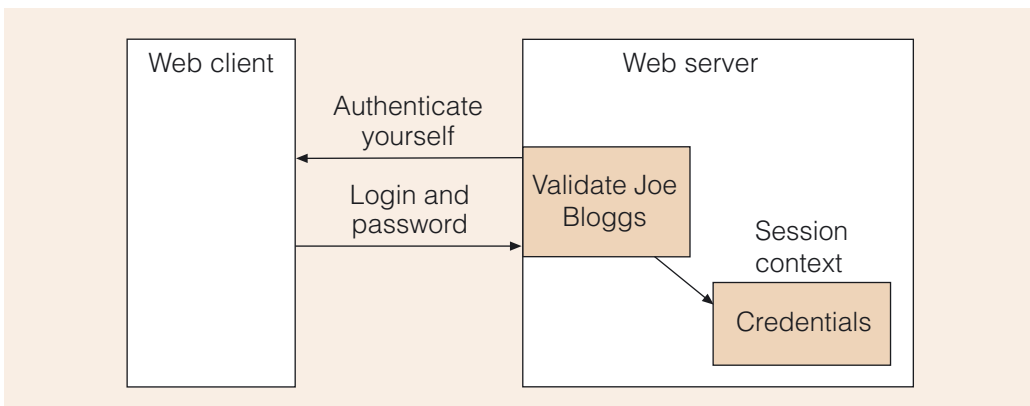


Figure 13 Authenticating the client and storing its credentials

The server must then validate the credentials received and store them for future use. Although JAAS is one route to authentication within a server component (and might be used by an application client), authentication is more often performed by a web component.

We also noted in Section 4 that at this point the server could authenticate itself to a client using a protocol such as SSL, in which the server presents a certificate. SSL can also be used to maintain confidentiality for credentials outside the container so that they are not transmitted in plaintext. In other words, requests and responses could be sent using HTTPS.

To validate the user's (or other caller) credentials, they must already be stored on the server side, so some initial setting up is required. For example, a new user may be required to set up an account. An enterprise system for managing this requirement may enforce some policies regarding the choice of usernames and passwords, preventing duplication and easily guessed passwords, or requiring password changes from time to time. Of course, other forms of credential information could also be held by the server.

Container authorisation

When deploying web resources, the decision must be made as to whether these should be accessible to all or only to certain callers. Some authorisation may be enforced by restricting access to certain URLs, based on information about the caller, using the web deployment descriptor. We will see how this is achieved in Subsection 7.2.

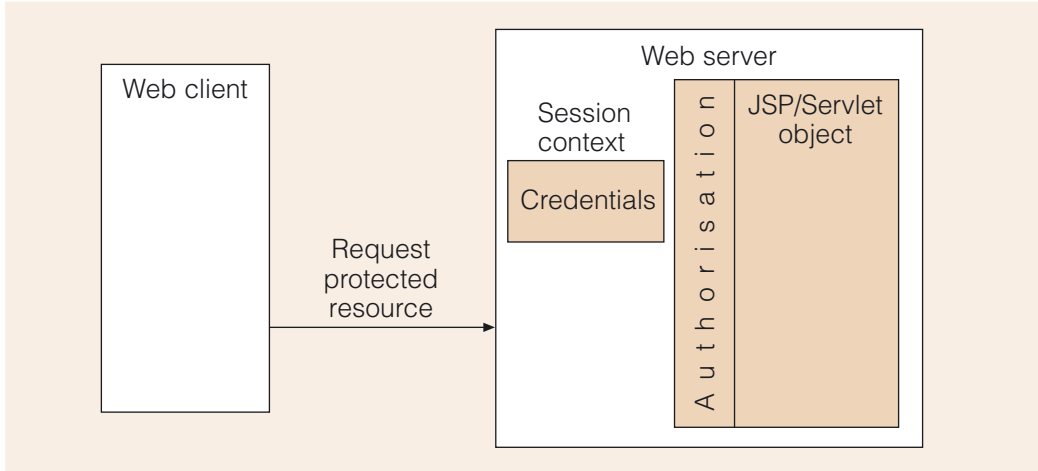


Figure 14 Requesting a protected resource (when credentials are stored)

Once a caller is authenticated and its credentials are stored in the session context, the context can be queried to determine who the caller is, and therefore whether any requested action should be authorised, using the container security policy.

Likewise, when writing EJB code, the programmer must consider whether methods should be accessible to all, or whether only certain callers should be allowed to execute the code. However, EJB components would not normally concern themselves with authentication. Rather they would assume that this task has been carried out elsewhere and that the identity of a caller is available within the EJB context. Based on this assumption, authorisation can be applied according to the caller's identity within the EJB.

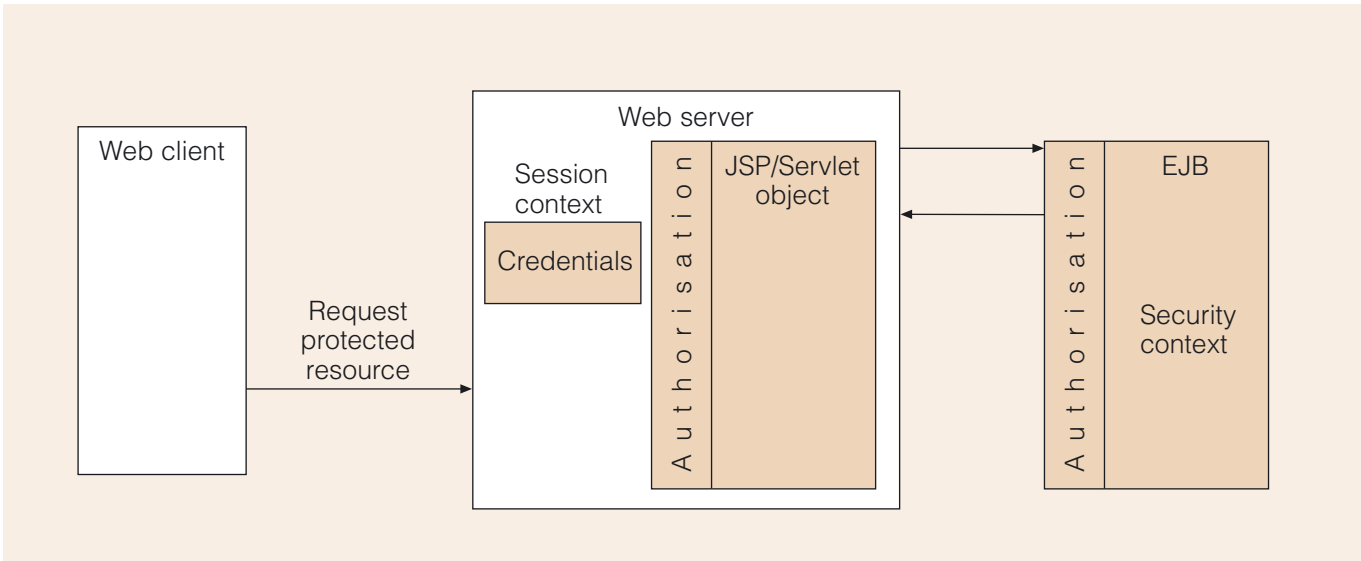


Figure 15 Accessing EJB resources via a web server

The following subsections discuss how these effects are achieved, including how callers are identified on the server, authentication mechanisms and authorisation mechanisms.

SAQ 18

What steps are taken when a protected web resource is accessed?

ANSWER.....

At this point (if lazy authentication is used) the caller must be authenticated. If successful, this results in the server storing credentials for the caller in a session context. The credentials are subsequently used to decide if the caller may access the protected resource, using a principal or role (see below for a discussion of roles).

7.2 Authentication mechanisms

Collections of users are stored on the server side in **realms**, and components are deployed within a particular realm. A realm may be implemented as a file of username and password pairs, for example, through a database or by using a naming service such as LDAP or JNDI. Web containers provide mechanisms to support interfacing with such external account systems.

Roles

We now explain in more detail how callers are represented on the server side.

Once authenticated, a caller can be identified by one or more principals, which are mapped to **roles** by the container. A role represents a type of user, for example, a customer or administrator. Setting up roles requires some initial configuration, but can be simplified by collecting principals in a **group** and assigning a role to the group. Belonging to a role is similar to having a key to a room containing the various resources the role is allowed to use – all members of the role have the same privileges.

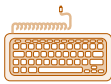
Remember that principals are created on authentication. Therefore, when a caller is authenticated, if the caller's principal or group can be mapped to a role permitted to access the requested resource (or code), then the action is authorised.

Roles separate the mechanism for controlling security from descriptions of users or callers outside the container. This means that the container need not be concerned with who or what has made a request, but only with whether or not the role associated with a caller is allowed to do what it is attempting. Notice also that this means that only the person deploying the code needs to know what roles users are in, and hence what privileges they have.

Provided mechanisms

In addition to any vendor-specific authentication scheme, containers may provide the following forms of authentication. Which method is to be used can be determined by configuring the container using its deployment descriptor.

- 1 HTTP basic authentication is based on a username and password, for example using a dialogue box. This is not secure, because data is transmitted in an easily decoded format (known as base-64).
- 2 Form-based authentication allows customisation of the user interface, for example using servlets.
- 3 HTTPS client authentication authenticates a client using a certificate using HTTP over SSL and so is more secure.
- 4 Mutual authentication requires both the client and the server to authenticate to each other using certificates or (for the client) login and password. It is rare to require a client to present a certificate, however.



Activity 10.8
Adding users to a domain.

It is also possible to add additional protection to basic or form-based authentication, for example using HTTPS in the transport layer and VPN in the network layer.

A web component can rely on several of these facilities (or other vendor-specific mechanisms) being available and use them to authenticate a caller.

SAQ 19

What is a role and how does it relate to users of a system?

ANSWER.....

A role is a name for a kind of caller and can be used to authorise actions. Users can be mapped to groups or roles so that the permissions applied to the roles apply to those users. Thus, a programmer does not have to know the users of a system, just the different roles of users of a system.



Activity 10.9

Web resource authorisation.

7.3 Authorisation mechanisms

There are three techniques to achieve authorisation in containers: declarative security, programmatic security and annotations. We discuss these approaches in the following three subsubsections.

Declarative security

In **declarative security**, the deployment descriptor file declares which roles are authorised to perform which actions; for example, access URLs or execute methods within an EJB. Each component has a deployment descriptor, such as `application.xml` for application clients, `sun-ejb-jar.xml` for EJBs or `web.xml` for web applications, to provide information about the component.

The deployment descriptor is read by the deployment tool to determine security requirements as well as various other options the container should use. As the descriptor file contents are external to the source code, the source code need not be altered when changes are made to the descriptor. In effect, this results in a delegation of security requirements to the container.

For example, two security roles could be defined within a deployment descriptor file as follows.

```
<security-role>
  <description>
    A Music Store administrator.
    This user can call the admin methods.
  </description>
  <role-name>Administrator</role-name>
</security-role>
<security-role>
  <description>
    A customer of the Music Shop.
    This role is allowed to access his/her own data.
  </description>
  <role-name>Customer</role-name>
</security-role>
```

This allows us to establish some known role names. Now we can also designate resources (for example, within a web server) and code (within an EJB) that should be made accessible to particular roles using a `<security-constraint>` element in the descriptor. For example, we can control access to a method within a particular EJB class `GeneralServices` as follows.

```
<security-constraint>
  <method-permission>
    <role-name>Administrator</role-name>
    <method>
      <ejb-name>GeneralServices</ejb-name>
      <method-name>emailSupport</method-name>
      <method-param>java.lang.String</method-param>
    </method>
  </method-permission>
</security-constraint>
```

This allows us to restrict access to the `emailSupport` method taking a `String` as an argument to the role `Administrator`. Rather than specifying individual method names, it is also possible to restrict access to all methods within a bean using a `*` notation within the `<method-name>` element.

Principals and groups are mapped to roles using the `<security-role-mapping>` element so that the application need not know the names of principals or groups.

Fortunately, we do not usually have to write this kind of description, as an implementation of Java EE should provide an environment that allows these properties to be set via a graphical interface. There may be occasions when interaction with raw XML is required, but they are increasingly rare.

Programmatic security

In **programmatic security**, a caller's identity is discovered through method calls on the context; for example, in EJBs there are methods such as `getCallerPrincipal` and `isCallerInRole`, which can be used to determine whether requested actions should be authorised.

Similar methods are available to web components via their contexts to determine who the caller is. In particular, an `HttpServletRequest` object has methods `getRemoteUser`, `isUserInRole` and `getUserPrincipal` so that decisions can be made in a servlet as to how to handle caller requests.

For example, the following illustrates how a caller's role could be used within an EJB method to determine whether code can be executed or not, assuming that `context` references the bean's context.

```
public void incrementSalary(Employee e)
{
    if (!context.isCallerInRole("accountant"))
        // throw new SecurityException(...);
    else
        // Increment employee's salary.
}
```

Also, as we saw in JAAS, containers provide methods you can use to perform code as a caller; for example, EJBs have a method `runAs` to run code as a certain role.

Annotations are overridden by settings in the deployment descriptor.

Programmatic security is more error-prone, as it depends on correctly implementing business logic in source code, while declarative security allows a more blanket approach to use of code, akin to object-oriented access levels such as private or public. Declarative security makes code more portable because any policies relating to roles must be described externally. Programmatic security is therefore mainly used when more fine-grained control is required; for example, when the policy to be enforced depends on a property that may change from one instance of a bean to another.

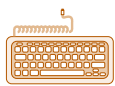
Annotations

Annotations are a relative newcomer to enterprise Java and incorporate aspects of declarative and programmatic security. Annotations are the recommended way of declaring roles.

The main security annotations are summarised in Table 6. Because some of these notations are not available in servlets, in those cases you must use a declarative style to achieve the same effects.

Table 6 Common security annotations

Annotation	Applied to	Used by	Description
@DeclareRoles	Class	EJB, servlet	Defines roles that can be used by programmatic checks
@DenyAll	Class, method	EJB	Deny access to designated methods to all users – this means that the methods can be used only outside a container
@PermitAll	Class, method	EJB	Give access to designated methods (or all methods in a class) to all users
@RolesAllowed	Class, method	EJB	Only users associated with the mentioned roles can access the designated methods
@RunAs	Class	EJB, servlet	Specifies the role to be used to run the designated component



Activity 10.10
EJB annotation.

The annotations @PermitAll, @DenyAll and @RolesAllowed can be applied to classes, in which case all methods in the class are affected, as well as to individual methods, in which case any class-level annotation is overridden. The permissions implied by these annotations are either inherited or overridden by annotations in a subclass.

For example, the notation @RolesAllowed can be used to list roles permitted to execute all methods in a class as follows.

```
@RolesAllowed({"Administrator", "Root-user"})
public class AdminFunctions
{
    // Class body.
}
```

This indicates that only callers in the roles "Administrator" or "Root-user" can execute any method in this class.

This annotation could also precede methods in the class. For example, you could annotate a `hello` method as follows.

```
@Stateless
public class HelloBean implements bean.HelloRemote
{
    @RolesAllowed("Customer")
    public String hello()
    {
        return "Hello valued customer!";
    }
}
```

There are some rules about how you can use these annotations in combination with one another; for example, you cannot both permit all and deny all.

SAQ 20

Explain the difference between declarative and programmatic security.

ANSWER.....

In declarative security, security roles and restrictions on the execution of code are established using entries in XML deployment descriptor files. In programmatic security, roles and restrictions are established using Java code.

8

Summary

Enterprise systems incorporate software, communication networks, peripherals and users, and these are key business assets that require protection. Keeping such systems secure requires consideration of all these levels as well as security management processes.

A computer system cannot be considered totally secure; however, attackers may choose a softer target if a system is well protected. Kinds of attack include modification, interruption, interception and fabrication.

To counteract these threats, enterprise systems require support for confidentiality, integrity, non-repudiation, auditing, authentication and authorisation. These services also help to ensure availability of the system.

Policies at both the system and user levels can be put into place; for example, to deploy firewalls, to help ensure software is kept up to date, to help enforce best practice in choice of passwords and to educate users about common risks.

Cryptography can be used to provide confidentiality and integrity of communications as well as non-repudiation, using a variety of mechanisms such as private key systems, public key systems and hashing. Block ciphers are deployed for bulk encryption, while stream ciphers are often used in open-ended communications such as wireless communication. The JCE packages are available to provide these services, allowing programmers to concentrate on the required cryptographic services such as digital signing, encryption or integrity checking.

Security of data over networks and of equipment connected to networks is an important consideration and providing this involves the use of secure protocols such as SSL and IPv6. The JSSE packages support SSL and TLS communications to provide security for communications outside enterprise containers.

Java and its virtual machine provide a secure programming platform on which to build, with common security holes such as buffer overflows or memory corruption issues being avoided by design. Load time and runtime checks are also employed to validate code as it is loaded into the virtual machine.

Fine-grained control can be exercised over authorisation for access to system resources and privileged code through security managers and system policies, based around permissions to perform actions. Security managers are an integral part of the containers in which enterprise system components such as servlets and EJBs are deployed. Permissions can be applied according to where code originated, what caller it is associated with or how it is signed.

JAAS provides a client- and server-side architecture supporting different forms of authentication in such a way that applications can be insulated from changes in authentication technology. Different authentication mechanisms can be layered on each other with a variety of stringency checks applied. Authorisation is applied according to principals associated with the code.

Containers also directly support authentication mechanisms such as basic, forms-based or certificate-based authentication.

Role-based authorisation is provided, meaning that there is a separation between the creation of user accounts and the privileges that users enjoy, and that a programmer

does not have to be concerned with who the users are, only with what certain groups of users may be permitted to do. Authorisation mechanisms can be applied to web resources and EJB code in this way. Security requirements in containers can be specified using a declarative or programmatic approach, each providing differing flexibility. Annotations provide a convenient form of declarative security within program code.

System security is a specialist field, to which we have given only a brief introduction (suggestions for further reading are contained on the M362 website). The nature of security services required changes with time. Thus, system security must be considered an ongoing concern.



Further reading.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- ▶ define standard security services;
- ▶ identify common threats and commonly implemented security policies;
- ▶ explain the differences between block ciphers, stream ciphers and hash functions and state some of their applications;
- ▶ use the Java Cryptography Extension (JCE) packages to deploy provided ciphers;
- ▶ discuss the principles behind public and secret key cryptography, strength of encryption and when to use one or the other form of cryptography (without reference to specific algorithms);
- ▶ discuss the principles of digital signing and certificates;
- ▶ discuss issues relating to secure communication and secure protocols;
- ▶ identify and discuss security-related features of the Java language and platform;
- ▶ discuss the Java Authentication and Authorization Service (JAAS);
- ▶ discuss authentication and authorisation mechanisms within enterprise component containers.

Glossary

access controller A component used by the security manager to check permissions belonging to classes.

action (in a policy file) In Java, the part of an entry in a policy file that describes the kinds of action that may be performed with respect to some target.

application domain A domain containing less privileged (non-system domain) classes in a protection domain.

application gateway A kind of firewall providing a proxy service.

asymmetric cryptography A code in which the encoding key is different from the decoding key, but is related to it in a non-trivial way (an example of this is **public key cryptography**).

audit log A record of the activity on a system that can be used to determine what users have done and to assist recovery in the case of an attack.

auditing Capturing a secure record of security-related events for non-repudiation, recovery and proof of effectiveness.

authentication The process of determining a user's or software's identity.

authorisation The process of determining what code is allowed to do, according to how it is signed, what code base it comes from or what caller it is associated with (for example, what user invoked it).

availability The property of a system that can be used when it is needed by an authorised user or application.

block cipher A cipher in which encryption takes place on blocks of plaintext.

brute-force attack An attack in which possible keys are attempted until a correct one is found.

buffer overflow The error that occurs when a buffer is assigned more data than it should hold.

certifying authority An organisation that issues a certificate associating a user with a cryptographic key.

challenge A method of authorisation in which the user is presented with a question or other task whose successful completion helps to determine their identity.

checksum A number used to check the integrity of a message.

cipher A code, a means of encrypting plaintext.

ciphertext The output of a cipher.

click fraud The fraudulent generation of clicks in order to profit from cost-per-click web pages.

code base The location of code in terms of its URL.

collision A collision occurs in a hash function when two or more inputs generate the same output.

confidentiality The property of a system that ensures that data is visible only to authorised users.

credentials Security information associated with a subject, such as keys or passwords.

cryptanalysis The science of decoding ciphers.

cryptographic hash function A hash function with desirable properties for applications in security.

cryptographic service A cryptographic facility provided by implementers of services for Java's cryptography API, such as ciphers, message digests and digital signing.

cryptography The science of ciphering.

deciphering Converting ciphertext to plaintext.

declarative security Using an XML deployment descriptor file to delegate authorisation checks to a container.

decryption See **deciphering**.

digital certificate An electronic means of associating a user with a key.

digital signature An electronic means of signing a communication.

digitally signed Having an electronic signature.

domain A set of classes with the same permissions.

end-to-end encryption An approach to network security in which the sending and receiving nodes of a message implement encryption and decryption using a shared key.

exponential Growing according to an exponential relationship; that is, according to a power of a number.

fabrication The construction of false or unauthorised information.

firewall A technology to filter internet traffic between a local and external network.

group A collection of authenticated users.

hacker An unauthorised user of a system.

handshake An exchange of information to establish parameters for communication.

hash function A one-way function that maps an input to a small output.

hierarchy of trust The relationship in which one or more parties place trust in other parties to perform authentication.

homograph attack An attack using social engineering in which a name similar to another is used to try to deceive a user.

identification See **authentication**.

integrity The property of information that it has not been altered by an attacker or otherwise corrupted.

interception Access of information by an unauthorised user.

interruption An attack preventing availability of a resource.

key A value used to decipher ciphertext.

key space The set of keys associated with a cipher.

key stream The generation of changing keys for the encryption of data in a stream cipher.

linear Growing according to a linear relationship, that is, by a constant factor for each fixed change in input.

link encryption An approach to network security in which each node implements encryption and decryption, each communicating pair using a different shared key.

linking The stage at which classes are incorporated into the runtime state of a virtual machine.

loading The stage at which classes are located and checked.

memory leak Loss of memory areas due to mismanagement of memory.

message authentication code (MAC) A code used to check the integrity (and in some cases identity) of a message.

message digest A reduced size version of a message produced by a hash function.

middle person An attacker interposed between a sender and receiver in location or time.

modification Unauthorised alteration of information.

nonce A value used once to defeat replay attacks.

non-repudiation The property of a system that prevents users from denying sending or receiving some information or performing some action.

one-time pad A cipher with perfect secrecy, invented in 1917.

one-way function A function with the property that it is easy to obtain an output from an input, but hard to determine the input given the output.

packet The unit of information in network communication.

packet filtering A technique allowing only desirable packets to travel across a firewall.

perfect secrecy The property of a code in which the ciphertext does not reveal any information about the plaintext or code.

permissions Values that determine which actions are authorised within a system.

plaintext Communication that has not been subjected to encryption.

pointer A memory address that may be manipulated in a variety of ways.

policy entry An entry in a policy file describing permissions assigned to a class according to its code base, signing or principals.

polyalphabetic substitution A code in which ciphertext is formed from groups of more than one character at a time.

primordial loader The trusted class loader in the JVM which bootstraps the system, loading the main class.

principal An identity associated with a subject, such as the subject's login ID or an organisation's name.

private key A cipher key that is kept secret and is used in conjunction with a public key in asymmetric cryptography.

programmatic security Explicit authorisation checks by method invocation.

protection domain The interaction of features enforcing security policies in Java, particularly as regards permissions allocated to classes.

provider A supplier of cryptographic products.

public key A key used in combination with a private key that may be publicised without compromising security of communication.

public key cryptography Cryptography in which public and private key pairs are employed; an example of **asymmetric cryptography**.

public key infrastructure (PKI) A system for the association of keys with users and the management of key-related information.

realm A set of valid users and groups for an application.

reference A memory value that cannot be manipulated, but merely provides access to objects.

replay An attack in which intercepted or stored information is reused to gain advantage.

role A collection of identities or category of identities with shared permissions.

root certificate A certificate granted to a certifying authority.

salt A number used to prevent collisions in hash functions and or in general to add variability to the processing of information.

secret key A key that must be kept secret, as used in symmetric cryptography.

secure channel A communication channel between a pair of processes providing authentication, confidentiality, integrity services and time stamping.

security manager (JVM) A component of the JVM concerned with authorisation of what code may or may not do.

security policy (JVM) A collection of permissions contained in a policy object, usually determined using policy files.

security through obscurity The discredited notion that secrecy of design alone can provide security.

session key A key that is used only while a session is in effect and then discarded.

singleton A programming design pattern ensuring that only one of an object may exist at a time.

social engineering A collection of techniques used to manipulate people into performing actions, or divulging confidential information. The term typically applies to trickery associated with information gathering or accessing computer systems.

stateful packet inspection A firewall technique in which the state of a system is used to determine the legality of traffic across a firewall.

statically typed Checking typing of variables at compile time.

statistical analysis A cryptanalytical technique using patterns of frequency of letters in languages as an aid to deciphering messages.

stream cipher A cipher whose key changes frequently to encrypt a stream of data.

strong typing Checking for potentially troublesome conversions of types at compile time.

subject (for authentication) A grouping of related information for an entity in a security system, such as a collection of identities (principals) and credentials.

substitution A ciphering technique in which one character in a language is substituted for another.

symmetric cryptography Cryptography employing a private key or in which the decoding key is trivially related to the coding key.

system In the context of security, refers to the hardware, platform, application software and users' interactions.

system domain The collection of classes within a protection domain that are trusted.

target An entity to which a permission is applied in a policy file.

threat monitoring The active consideration of dangers to a system's security.

time stamping The inclusion of information with a message so that its time of occurrence can be determined.

transposition An encryption technique where the order of information in the plaintext is obscured.

trapdoor function A one-way function which is reversible if given some secret information.

tunnelling Wrapping one protocol in another protocol.

type safety The ability of a language to check that types are being used correctly.

vector A route by which an attack may occur on a computer system.

virtual private network (VPN) A network created by a form of tunnelling in which end-to-end encryption takes place between the two ends of the tunnel, so creating a private network of sorts.

web of trust A non-hierarchical approach to establishing trustworthiness of digital certificates.

References

- Diffie, W. and Hellman, M.E. (1976) 'New Directions in Cryptography', *IEEE Transactions on Information Theory*, vol. IT-22, no. 6, pp. 644–654.
- Hunter, R. (2003) *Enterprises and Employees: The Growth of Distrust*, The Gartner Group.
- International Organization for Standardization (2005) *Information Technology – Security Techniques – Code of Practice for Information Security Management ISO/IEC 17799*, Geneva, Switzerland, International Organization for Standardization.
- Lemos, R. (2002) *What are the Real Risks of Cyberterrorism?* [online], http://news.zdnet.com/2100-1009_22-955293.html (Accessed 19 October 2007).
- Shannon, C.E. (1949) 'Communication theory of secrecy systems', *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715.
- Stallings, W. (1999) *Cryptography and Network Security*, Prentice Hall.

Index

A

- access control 11, 46
 - role-based 51
- action (in a policy file) 48
- application
 - domain 46
 - gateway 38
- attack
 - brute-force 16
 - denial of service 10
 - homograph 9
 - man-in-the-middle 10
 - middle person 10
 - person-in-the-middle 10
 - replay 10
- auditing 11
- authentication 11
 - challenge approach 26
 - hierarchy of trust 26
 - JAAS 51
- availability (of a system) 11

B

- buffer overflow error 41
- bytecode verifier 43

C

- certificate
 - certifying authority 28
 - digital 28
 - root 28
 - web of trust 29
- checksum 30, 35
- cipher 14
 - block 17–18
 - Caesar 15
 - exponential 22
 - rail-fence 16
 - stream 31–32
 - Vernam 31
- ciphertext 14
- class loader 42
- click fraud 8
- code base 45
- collision (input table) 29

- confidentiality 11
- container security 58
- credentials 51
- cryptanalysis 16
- cryptographic
 - hash function 29
 - salt 30
 - service 14
- cryptography 14
 - asymmetric 22
 - public key 22
 - secret key 21
 - statistical analysis 16

D

- deciphering 14
- declarative security 62
- decryption 14–15
- digital signature 26
- domain 45

E

- encryption 15, 21
 - end-to-end 34
 - link 34
 - substitution 15
 - symmetric 21
 - transposition 16
 - encryption algorithm
 - ARCFOUR 32
 - Rivest Cipher 4 (RC4) 32
 - RSA 24
 - encryption protocol
 - Diffie–Hellman 33
 - Pretty Good Privacy (PGP) 29, 33
 - Wi-Fi Protected Access (WPA) 32
 - Wireless Equivalency Protocol (WEP) 32
 - Enigma machine 17
 - exponential relationship 19
- ## F
- firewall 37
- ## G
- group 61

H

hacker 8
handshake 36
hash function 26, 29
honeypot 12

I

identification 11
integrity (of data) 11
Internet Protocol (IP) 35

J

Java Authentication and Authorization Service (JAAS) 51–52
Java Cryptography Architecture (JCA) 14
Java Cryptography Extension (JCE) 14, 51
Java Secure Socket Extension (JSSE) 37, 51

K

key 15
 private 23
 public 23
 secret 21
 session 20
 space 16
 stream 32

L

linear relationship 19
linking 43
loading 42

M

memory leak 41
message authentication code (MAC) 30
message digest 26
monitoring 11

N

namespace (of a Java class) 42
non-repudiation 11
nonce value 20

O

offset value 15
one-time pad 32
one-way function 24

P

packet 11
 filtering 38
perfect secrecy 32
permissions 45, 47
plaintext 14
Pluggable Authentication Module (PAM) 53
pointer 41
policy entry 48
polyalphabetic substitution 17
primordial loader 42
principal 46, 51
programmatic security 63
protection domain 45
provider 14
public key infrastructure (PKI) 28

R

realm 61
reference (Java pointer) 41
role-based access control 61

S

secure channel 34
Secure Sockets Layer (SSL) 36
security manager (JVM) 46
security policy (JVM) 44, 46
security policy (organisational) 11
security threat 9
 fabrication 10
 interception 10
 interruption 10
 modification 10
security through obscurity 12
SecurityManager 44, 58
singleton 44
social engineering 9
stateful packet inspection 38
subject (for authentication) 51
system 6
system domain 46

T

target (policy file) 48

threat monitoring 11

time stamping 30

Transport Layer Security (TLS) 37

trapdoor function 24

trust 26

tunnelling 39

type safety 40

statically typed 40

strongly typed 40

V

vector 8

virtual private network (VPN) 39

